

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

**Отчет о программном проекте на тему:
Алгоритмы предсказания цены и оптимизации**

(промежуточный, этап 1)

Выполнили студенты:

группы #БПМИ234, 3 курса
группы #БПМИ234, 3 курса
группы #БПМИ2310, 3 курса

Шадрин Андрей Романович
Климов Демьян Дмитриевич
Барсуков Борис Михайлович

Принял руководитель проекта:

Мунерман Илья Викторович
Руководитель Интерфакс-ЛАБ, к.э.н.

Москва 2026

Содержание

Аннотация	4
Ключевые слова	4
Введение	5
Описание предметной области	5
Цели и задачи	5
Результаты работы и их новизна	6
Структура всей работы	7
Разделение задач по участникам	7
1 Обзор литературы	8
2 Описание общего пайплайна	8
3 Работа с данными	10
3.1 Данные	10
3.2 Предобработка данных и создание признаков	10
4 Методы построения эмбедингов	12
4.1 TS2Vec Embedder	12
4.2 Handmade Embedder	13
5 Кластеризация эмбедингов	14
5.1 KMeans	15
5.1.1 Визуализация и анализ кластеров	15
5.1.1.1 Метрики анализа кластеров	15
5.1.1.2 TS2Vec	16
5.1.1.3 Handmade	20
5.1.1.4 Сравнение TS2Vec и Handmade	23
5.2 HDBSCAN	23
6 Модели прогнозирования доходности	23
6.1 Подбор гиперпараметров	24
6.2 ARIMA	24
6.2.1 Архитектура	24
6.3 Реализация	24
6.4 MR-AR	25
6.4.1 Архитектура	25
6.4.2 Реализация	26
6.5 LightGBM	27
6.5.1 Архитектура	27
6.5.2 Реализация	27
6.6 LSTM	29
6.6.1 Архитектура	29
6.6.2 Реализация	30
6.7 GRU	31
6.7.1 Архитектура	31

6.7.2	Реализация	31
6.8	CNN	32
6.8.1	Архитектура	32
6.8.2	Реализация	32
6.9	Transformer	34
6.9.1	Архитектура	34
6.9.2	Реализация	34
6.10	Метрики оценки	35
7	Методология тестирования	37
7.1	Данные для моделей	37
8	Результаты	37
8.1	Оценка качества предсказательной способности моделей	37
8.2	Модели без кластеризации	37
8.3	Модели с кластеризацией	39
8.3.1	TS2Vec + KMeans	39
8.3.1.1	C0 — нейтральный (боковое движение)	40
8.3.1.2	C1 — бычий тренд	40
8.3.1.3	C2 — высокая волатильность	41
8.3.1.4	C3 — медвежий тренд	42
8.3.1.5	C4 — аномальные движения	42
8.3.2	Handmade + KMeans	43
8.3.2.1	C0 — нейтральный (боковое движение)	43
8.3.2.2	C1 — бычий тренд	44
8.3.2.3	C2 — нисходящий с отрицательной асимметрией	45
8.3.2.4	C3 — кризисный режим	45
8.3.3	Промежуточные итоги и выводы	46
	Список литературы	47

Аннотация

В работе рассматривается задача прогнозирования доходности акций на Московской бирже и формирования на этой основе инвестиционного портфеля. В ходе исследования была проведена большая работа по сравнительному анализу уже известных моделей предсказания доходности, таких как LSTM, GRU, CNN, Transformer, ARIMA, MR-AR, LightGBM, а также были имплементированы свои алгоритмы предсказания доходности с использованием эмбедингов и кластеризации.

Оказалось, что наиболее эффективными оказались алгоритмы, разбивающие окна акций на кластеры и предсказывающие доходность в зависимости от кластера.

Помимо этого для каждого алгоритма была построена оптимизация портфеля по Маркоцу, а также был проведён бэктестинг на исторических данных.

Ключевые слова

прогнозирование цен, Московская биржа, формирование портфеля, алгоритмическая торговля, бэктестирование, машинное обучение, эмбединги, кластеризация

Введение

Описание предметной области

Прогнозирование цен акций — одна из центральных задач количественных финансов, привлекающая внимание как академического сообщества, так и практикующих участников рынка. Российский фондовый рынок (Московская биржа, MOEX) отличается рядом специфических черт: высокой концентрацией в сырьевых и финансовых секторах, неравномерной ликвидностью инструментов, чувствительностью к геополитическим факторам, — что делает задачу прогнозирования ценовой динамики особенно нетривиальной.

Традиционные подходы — технический анализ, классические модели временных рядов (ARIMA) — слабо адаптируются к смене рыночных режимов. Методы глубокого обучения (LSTM, Transformer, CNN) повышают качество прогноза, однако, как правило, обучаются независимо для каждого инструмента и не используют межинструментальную информацию о схожести ценовых паттернов. Ещё одна системная проблема — нестационарность: паттерны, эффективно работавшие в прошлом для конкретной бумаги, со временем деградируют. При этом аналогичные паттерны могут проявляться в других инструментах или в другие временные периоды.

Настоящая работа предлагает альтернативный подход. Вместо прямого предсказания цены или доходности для каждого инструмента отдельно мы сначала отображаем скользящие окна ценовых рядов всех акций в единое латентное пространство (получаем эмбединги), после чего кластеризуем эти представления и используем кластерную структуру как основу для адаптивного прогнозирования и оптимизации портфеля по Марковицу. Гипотеза состоит в том, что близость в латентном пространстве отражает содержательное сходство ценовой динамики вне зависимости от конкретного инструмента и момента времени — и именно этот перенос знаний между инструментами и периодами позволяет строить более робастные торговые стратегии.

Цели и задачи

1. Рассмотреть базовые алгоритмы предсказания временных рядов (LSTM, GRU, CNN, Transformer, ARIMA, MR-AR и LightGBM), сравнить их и выбрать лучшие на данных MOEX.
2. Разработать свои собственные подходы к предсказанию доходностей акций на Московской бирже.
3. Провести сравнительный анализ всех моделей по метрикам качества
4. Построить оптимизацию портфеля по Марковицу и провести бэктестирование на исторических данных MOEX.

В качестве базовых моделей сравнения рассматриваются: LSTM, GRU, CNN, Transformer, ARIMA, MR-AR и LightGBM — каждая из которых имеет свои преимущества и недостатки в контексте финансовых временных рядов.

Фундаментальное ограничение большинства существующих подходов состоит в том, что модель прогнозирования обучается на всей совокупности данных сразу, усредняя поведение рынка через все режимы и инструменты. Такая модель вынуждена искать глобальные закономерности, жертвуя точностью на локальных паттернах. Между тем рынок нестационарен: паттерн, новый для конкретной бумаги в данный момент, вполне

мог встречаться ранее в других акциях или в другие временные периоды. Иными словами, перенос знаний между инструментами и периодами потенциально ценен, но стандартными методами не используется.

Предлагаемый в работе подход строится на следующей идее. Весь массив ценовых рядов рассматривается как единая совокупность скользящих окон фиксированной длины. Каждое окно отображается в латентное пространство — получается эмбединг, компактно описывающий форму ценового движения. Близкие в этом пространстве окна образуют кластеры, соответствующие схожим рыночным режимам или каким-то определенным паттернам. Для каждого кластера затем выбирается и обучается наилучшая прогнозная модель, способная более точно уловить специфику именно этого типа динамики.

Ключевым шагом является качественное извлечение признаков из окон: эмбединги должны сближать окна с похожей динамикой и разводить окна с различным поведением. Применение эмбедингов позволяет, во-первых, абстрагироваться от масштаба цен отдельных акций, а во-вторых, выявлять повторяющиеся паттерны поведения, которые могут соответствовать определённым рыночным режимам или фазам движения. В работе рассматриваются и сравниваются два принципиально разных подхода к построению эмбедингов:

- **Обучаемые эмбединги (TS2Vec)** — нейросетевой кодировщик (контрастивное обучение) выявляет скрытые межоконные зависимости, которые могут быть неочевидны или вовсе неизвестны заранее.
- **Рукотворные эмбединги (Handmade)** — векторы из интерпретируемых статистических признаков окна (волатильность, наклон тренда, асимметрия и др.), не требующие обучения модели.

После того как эмбединги построены, можно уже использовать кластеризацию. Таким образом, мы получаем разбиение окон всех акций на кластеры, которые соответствуют схожим паттернам.

Результаты работы и их новизна

В ходе работы был разработан и реализован пайплайн, включающий обучение эмбедингов ценовых окон, кластеризацию рыночных паттернов, кластерно-адаптивное прогнозирование доходности и оптимизацию портфеля по Марковицу с бэктестированием на данных МОЕХ. При этом подтвердилась гипотеза о том, модели заточенные под свой конкретный кластер работают лучше чем модели, обученные на всех данных сразу.

Это результат позволяет строить прогнозные системы, явным образом учитывающие текущий рыночный паттерны (не просто режимы). Это повышает устойчивость портфельных стратегий к смене паттерна рынка и снижает зависимость от глобальной стационарности данных — свойство особенно важное для российского рынка с его высокой чувствительностью к внешним шокам. Таким образом, теперь нам достаточно понять к какому кластеру относится окно конкретной акции, да бы сделать качественный прогноз.

Научная же польза проекта состоит в том, что всё исследование проводилась на данных МОЕХ, поскольку работ, использующих Московскую юиржу крайне мало.

Структура всей работы

Раздел 1 описывает общий пайплайн работы и его этапы, которые разделены по следующим разделам:

- Раздел 2 (работа с данными): описываются типы данных, сколько их всего, создание новых фичей, а также рассказывается как данные разбиваются на обучающую, валидационную и тестовую выборки.
- Раздел 3 (методы построения эмбеддингов): описываются два метода построения эмбеддингов: TS2Vec (какого размера и как обучаются) и Handmade (какие фичи формируются по окну).
- Раздел 4 (кластеризация эмбеддингов): описываются два метода кластеризации: k-means и hdbscan. Как они применялись на эмбеддингах, интерпретация кластеров, а также рассказывается о неудачном эксперименте с hdbscan.
- Раздел 5 (модели прогнозирования доходностей): детально описывает базовые модели, подбор перебор каких гиперпараметров осуществлялся и как.

В разделе 6 детально описываются результаты работы. Рассказывает о том какие метрики получились у моделей на каждом кластере, а также какие метрики получились у моделей без кластеризации. Проводится сравнение всех моделей по всем метрикам. А также описываются с какими гиперпараметрами запускались модели.

Разделение задач по участникам

1. Борис Барсуков:

- Получение данных с МОЕХ берже (работ с БД)
- Оптимизация портфеля по Марковицу
- Проведение бектестирования всех алгоритмов на исторических данных
- обучения всех базовых моделей и проведение анализа

2. Шадрин Андрей:

- Построения TS2Vec эмбеддингов
- Кластеризация эмбеддингов (k means, hdbscan)
- Интерпретация получившихся кластеров
- На получившихся кластерах провести обучения и подбор гиперпараметров базовых моделей
- Выявить лучшую модель на каждом кластере

3. Климов Демьян:

- Построение Handmade эмбеддингов, выбор статистик
- Кластеризация эмбеддингов (k means, hdbscan)
- Интерпретация получившихся кластеров
- На получившихся кластерах провести обучения и подбор гиперпараметров базовых моделей
- Выявить лучшую модель на каждом кластере

1 Обзор литературы

2 Описание общего пайплайна

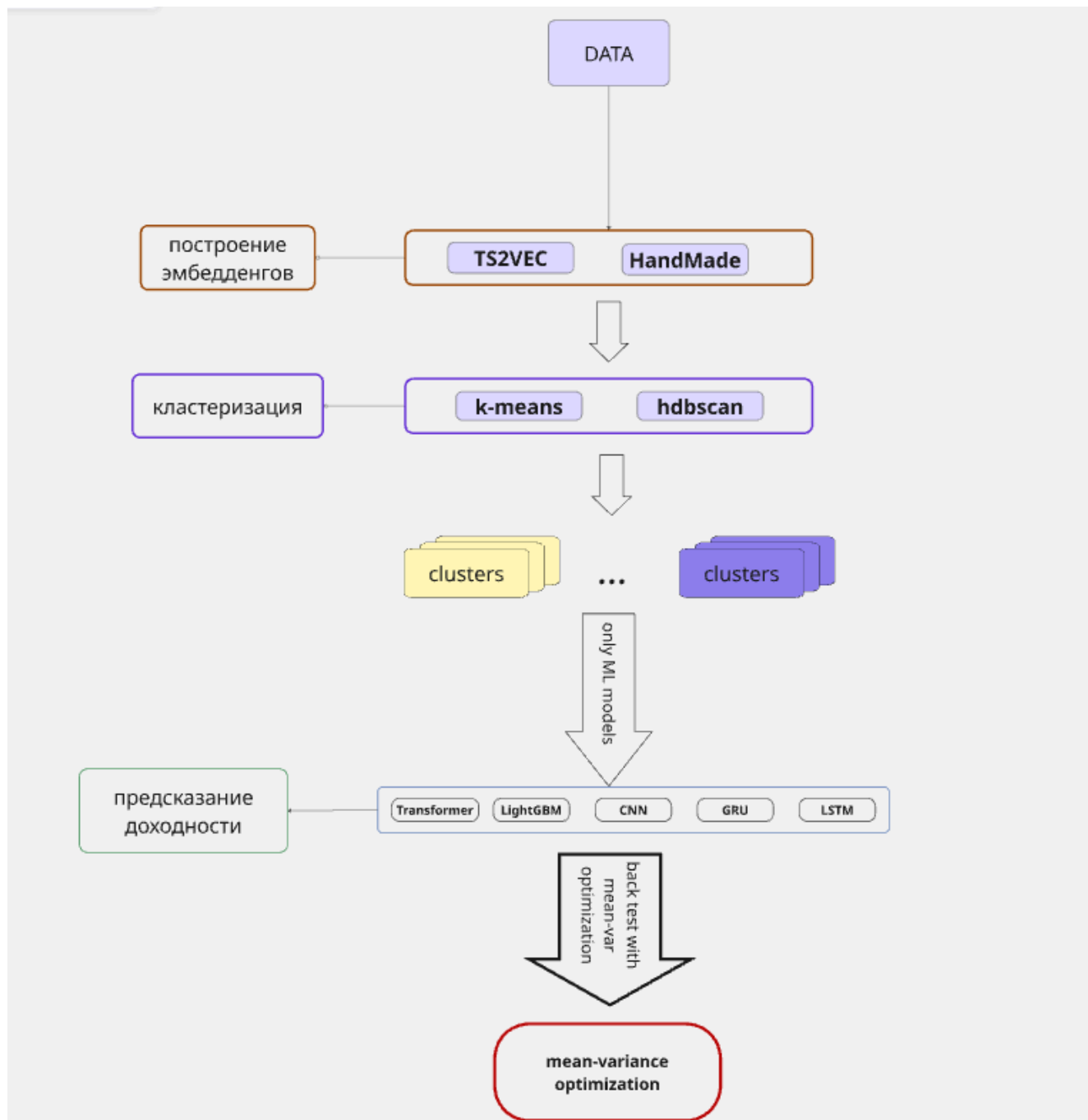


Рис. 2.1. Наш подход

Наша гипотеза состоит из нескольких подходов: построение эмбеддингов, затем кластеризация эмбеддингов, а затем прогнозирование доходностей на каждом кластере. Причем для каждого из этапов нам необходимо искать оптимальное решение. Т.е. нам необходимо проверить с каким алгоритмам кластеризации наши эмбеддинги работают лучше всего: hdbscan или k-means, сравнить их по метрикам и найти лучший, а также подобрать оптимальное число кластеров, чтобы было достаточно данных для обучения, но и кластеры не сливались в кучу. А далее для каждого кластера нам необходима перебрать все модели и

посмотреть какая получит наилучший результат. Итогово мы определим лучшее разбиение по кластерам и лучшую модель в каждом кластере.

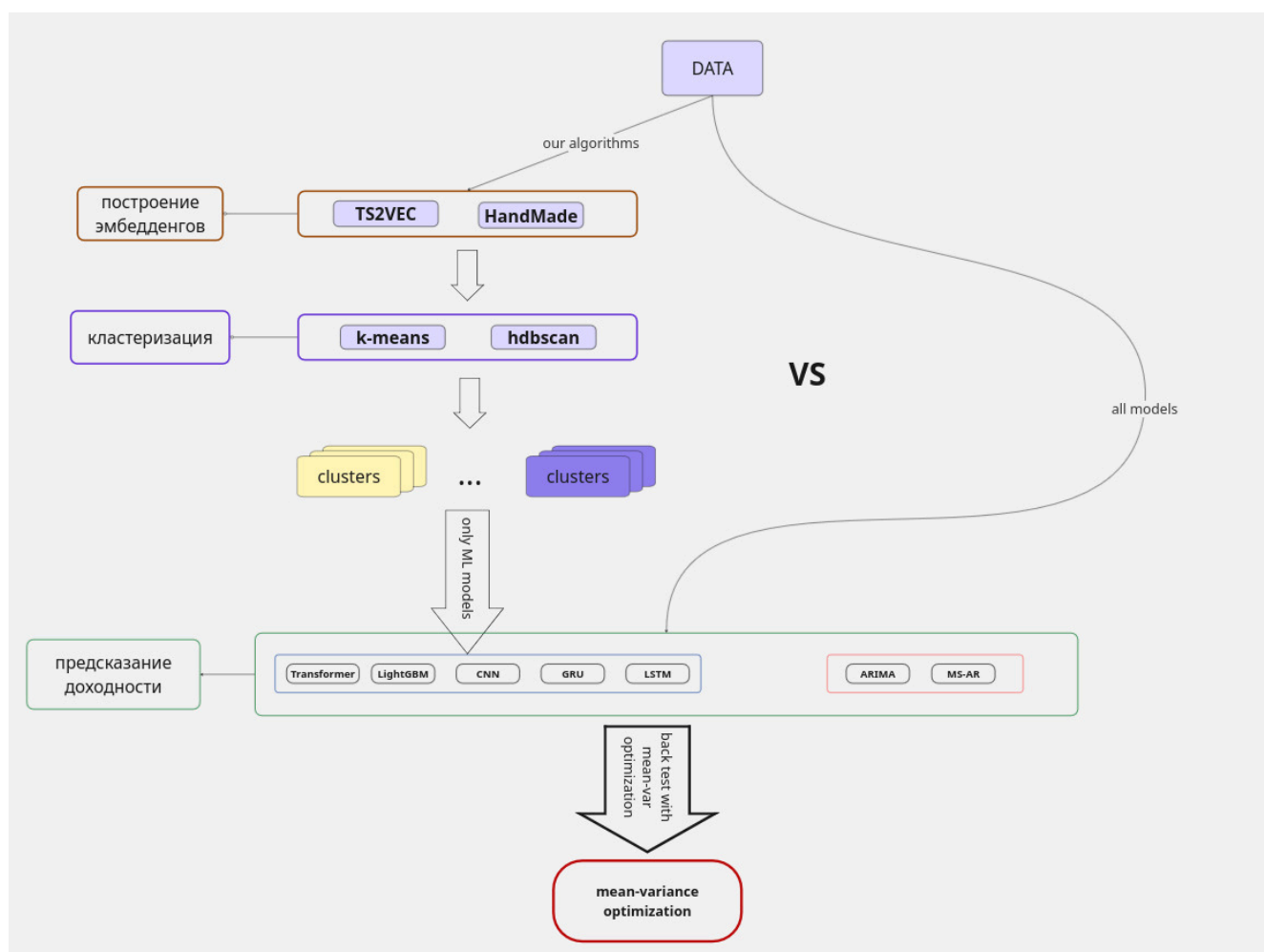


Рис. 2.2. Общая схема пайплайна

Далее необходимо провести детальное сравнения с базовыми моделями, обучив их на всех акциях сразу и провести бек-тестинг.

3 Работа с данными

3.1 Данные

В работе используются дневные данные о свечах всех валидных акций Московской биржи. Свечи базово включают 5 основных параметров помимо timestamp:

open	high	low	close	volume
------	------	-----	-------	--------

Где **open** - цена открытия свечи, **high** - максимальная цена свечи, **low** - минимальная цена свечи, **close** - цена закрытия свечи, **volume** - объём торгов.

Эти данные были собраны за период с начала существования акции на бирже вплоть до 2026 года.



Рис. 3.3. Дневные свечи акции SBER

3.2 Предобработка данных и создание признаков

Перед добавлением признаков в данные были удалены пропуски, а также исключены все акции, у которых было менее 252 торговых дней.

Далее для каждого временного ряда были добавлены следующие признаки:

sma5	sma20
ema12	ema26
close_sma20	macd
macd_signal	macd_hist
rsi14	bb_pct
bb_bw	atr14
obv	

Где признаки вычисляются по формулам:

- $sma5_t = \frac{1}{5} \sum_{i=0}^4 close_{t-i}$ - средняя цена за 5 дней
- $sma20_t = \frac{1}{20} \sum_{i=0}^{19} close_{t-i}$ - средняя цена за 20 дней
- $ema12_t = \alpha_{12} close_t + (1 - \alpha_{12}) ema12_{t-1}$, $\alpha_{12} = \frac{2}{12+1}$ - экспоненциальная средняя цена за 12 дней
- $ema26_t = \alpha_{26} close_t + (1 - \alpha_{26}) ema26_{t-1}$, $\alpha_{26} = \frac{2}{26+1}$ - экспоненциальная средняя цена за 26 дней
- $close_sma20_t = \frac{close_t}{sma20_t}$ - отношение цены к средней цене за 20 дней
- $macd_t = ema12_t - ema26_t$ - разница между экспоненциальными средними за 12 и 26 дней
- $macd_signal_t = ema9(macd_t)$ - экспоненциальная скользящая средняя за 9 дней от $macd_t$
- $macd_hist_t = macd_t - macd_signal_t$ - разница между $macd_t$ и $macd_signal_t$
- $rsi14_t = 100 - \frac{100}{1+RS_t}$, $RS_t = \frac{avgGain14_t}{avgLoss14_t}$ - индекс относительной силы за 14 дней
- $sigma20_t = std(close_{t-19}, \dots, close_t)$ - стандартное отклонение цены за 20 дней
- $bb_upper_t = sma20_t + 2 \times sigma20_t$, $bb_lower_t = sma20_t - 2 \times sigma20_t$ - верхняя и нижняя границы полос Боллинджера
- $bb_pct_t = \frac{close_t - bb_lower_t}{bb_upper_t - bb_lower_t}$ - относительное положение цены внутри полос Боллинджера
- $bb_bw_t = \frac{bb_upper_t - bb_lower_t}{sma20_t}$ - ширина полос Боллинджера
- $TR_t = \max(high_t - low_t, abs(high_t - close_{t-1}), abs(low_t - close_{t-1}))$ - истинный диапазон
- $atr14_t = ema14(TR_t)$ - экспоненциальная средняя истинного диапазона за 14 дней
- $obv_t = obv_{t-1} + sign(close_t - close_{t-1}) \times volume_t$ - накопленный баланс объема торгов

Где $close_t$, $high_t$, low_t , $volume_t$ — значения цены закрытия, максимума, минимума и объема в день t соответственно.

Таким образом, всего получается 18 признаков, на которых обучаются модели.

open	high
low	close
volume	sma5
sma20	ema12
ema26	close_sma20
macd	macd_signal
macd_hist	rsi14
bb_pct	bb_bw
atr14	obv

Все данные были разделены на три непересекающихся временных выборки: тренировочную, валидационную и тестовую. Разбиение выполнено по дате, а не случайно, чтобы исключить утечку данных из будущего в прошлое.

Поскольку часть акций появилась на МОЕХ лишь в 2014–2015 годах, тренировочная выборка охватывает наибольший временной промежуток.

Выборка	Период	Доля
Train	2000 — 2022	5/7
Validation	2023 — 2024	1/7
Test	2025 — март 2026	1/7

4 Методы построения эмбеддингов

Кластеризация временных окон возможна только после приведения их к единому векторному формату. Поэтому для каждого окна строится эмбеддинг — числовое представление, сохраняющее информацию о структуре и динамике временного ряда.

Для построения эмбеддингов были использованы следующие методы:

- Handmade Embedder — метод построения эмбеддингов на основе ручных статистических признаков.
- TS2Vec Embedder — метод построения эмбеддингов на основе самообучаемой модели TS2Vec.

4.1 TS2Vec Embedder

TS2Vec [1] — unsupervised метод построения эмбеддингов временных рядов на основе иерархического контрастного обучения.

На каждой итерации модель получает окно временного ряда и порождает две аугментации: случайно обрезает начало/конец подпоследовательности и маскирует отдельные временные шаги.

Обе аугментации подаются в один и тот же энкодер, после чего вычисляется контрастная функция потерь: эмбеддинги одного и того же момента времени из разных аугментаций притягиваются, а эмбеддинги разных моментов — отталкиваются.

Энкодер состоит из проекционного слоя в размерность `hidden_dim` и `depth` дилатированных свёрточных блоков (TCN).

После энкодера представления проецируются в пространство размерности `output_dim`.

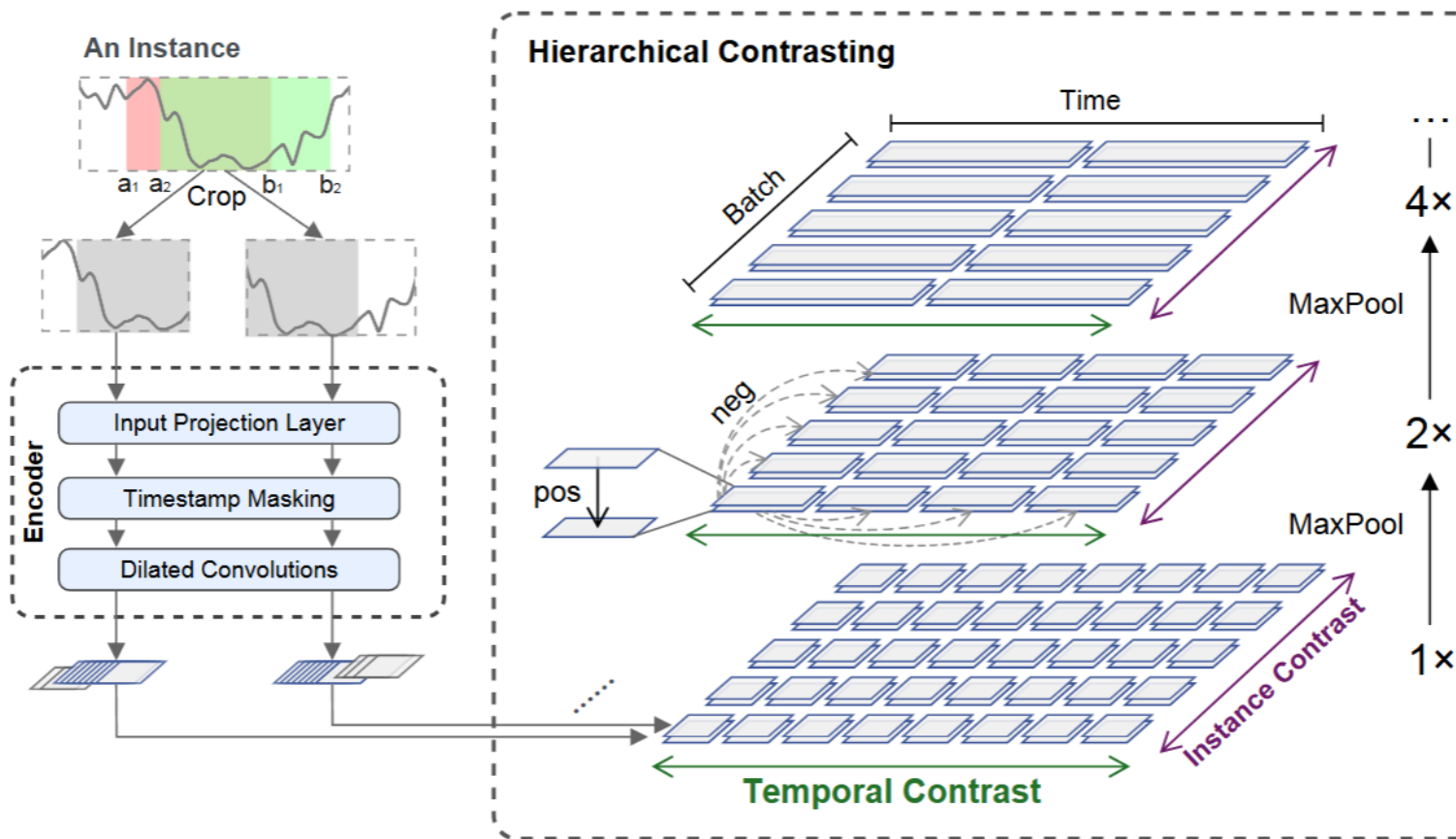


Рис. 4.4. Схема архитектуры TS2Vec

Для построения эмбеддингов был выбран `output_dim = 50` для окон длиной 60 дней. На обучающей выборке выполнен перебор гиперпараметров энкодера `hidden_dim` и `depth` с помощью Optuna.

Параметр	Значение
<code>hidden_dim</code>	128
<code>depth</code>	8
<code>output_dim</code>	50

Таблица 4.1. Гиперпараметры TS2Vec после подбора Optuna

4.2 Handmade Embedder

Handmade Embedder — детерминированный метод построения эмбеддингов, не требующий обучения. Для каждого окна длиной $T = 60$ дней вычисляется вектор из 15 признаков, который и является его эмбеддингом.

Обозначим z_t — z -нормированное значение close в момент t , v_t — volume, T — длина окна.

Трендовые признаки

- $\text{slope} = \frac{\sum_{t=1}^T (t-\bar{t})(z_t - \bar{z})}{\sum_{t=1}^T (t-\bar{t})^2}$ — линейный наклон ряда
- $\text{net_move} = z_T - z_1$ — суммарное ценовое движение от начала к концу окна

- $\text{frac_above_mean} = \frac{1}{T} \sum_{t=1}^T \mathbb{1}[z_t > \bar{z}]$ — доля баров, на которых цена выше среднего, значение 0.5 соответствует боковому, близкое к 0 или 1 — устойчивому тренду
- $\text{price_range} = \max_t z_t - \min_t z_t$ — размах ценового движения внутри окна

Волатильность

- $\text{daily_vol} = \text{std}(z_t - z_{t-1}, t = 2 \dots T)$ — стандартное отклонение дневных приращений
- $\text{skewness} = \frac{1}{T} \sum_{t=1}^T \left(\frac{z_t - \bar{z}}{\sigma_z} \right)^3$ — асимметрия распределения значений close, положительный скос указывает на редкие резкие подъёмы, отрицательный — на резкие падения
- $\text{kurtosis} = \frac{1}{T} \sum_{t=1}^T \left(\frac{z_t - \bar{z}}{\sigma_z} \right)^4 - 3$ — эксцесс, высокие значения указывают на тяжёлые хвосты и склонность к выбросам

Объёмные признаки

- $\text{price_vol_corr} = \frac{\sum_{t=1}^T \tilde{z}_t \tilde{v}_t}{\sqrt{\sum_{t=1}^T \tilde{z}_t^2 \cdot \sum_{t=1}^T \tilde{v}_t^2 + \varepsilon}}$ — корреляция цены и объёма, где $\tilde{z}_t = z_t - \bar{z}$, $\tilde{v}_t = v_t - \bar{v}$, положительные значения означают, что рост цены подтверждается объёмом
- $\text{vol_activity} = \text{std}(v_t - v_{t-1}, t = 2 \dots T)$ — стандартное отклонение приращений объёма, высокие значения характерны для периодов аномальной торговой активности

Снимок индикаторов в конце окна

- $\text{last_rsi} = \text{RSI}_T$ — значение RSI в последний день окна, отражает текущую перекупленность или перепроданность.
- $\text{last_macd_hist} = \text{MACD_hist}_T$ — значение MACD-гистограммы в конце окна, знак указывает на направление импульса
- $\text{last_bb_pct} = \text{BB_pct}_T$ — значение BB%b в конце окна; показывает, где цена находится относительно полос Боллинджера.
- $\text{last_bb_bw} = \text{BB_bw}_T$ — ширина полос Боллинджера в конце окна; высокие значения соответствуют высокой волатильности.

Наклон индикаторов внутри окна

- $\text{rsi_slope} = \text{RSI}_T - \text{RSI}_1$ — изменение RSI от начала к концу окна, положительные значения указывают на нарастание импульса
- $\text{macd_slope} = \text{MACD_hist}_T - \text{MACD_hist}_1$ — изменение MACD-гистограммы, показывает, усиливается или затухает текущий импульс

Итоговый вектор эмбединга имеет размерность 15. Все нечисловые значения (NaN, Inf) заменяются нулём.

5 Кластеризация эмбедингов

Кластеризация представляет собой процесс группировки объектов (в нашем случае — эмбедингов) на основе их схожести.

Для кластеризации были использованы следующие методы:

- KMeans — метод кластеризации на основе k -средних.

- HDBSCAN — иерархический плотностной метод кластеризации.

5.1 KMeans

KMeans — метод, который принимает на вход k — количество кластеров и определяет центроиды кластеров.

На каждой итерации алгоритм вычисляет расстояние от каждого объекта до каждого центроида и относит объект к ближайшему кластеру, после чего пересчитывает центроид как среднее арифметическое координат всех объектов кластера. Этот процесс повторяется до тех пор, пока центроиды не стабилизируются.

Подбор оптимального числа кластеров проводился по метрике **silhouette** (коэффициент силуэта).

Пусть $d(i, j)$ — расстояние между объектами i и j в пространстве эмбедингов. Обозначим через $C(i)$ кластер, которому принадлежит объект i .

Внутрикластерная средняя дистанция:

$$a(i) = \frac{1}{|C(i)| - 1} \sum_{j \in C(i), j \neq i} d(i, j) \quad (5.1)$$

Минимальная средняя дистанция до «чужого» кластера:

$$b(i) = \min_{C \neq C(i)} \frac{1}{|C|} \sum_{j \in C} d(i, j) \quad (5.2)$$

Силуэт точки i :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (5.3)$$

Средний силуэт по выборке из n объектов:

$$\text{Silhouette} = \frac{1}{n} \sum_{i=1}^n s(i) \quad (5.4)$$

Чем ближе значение к 1, тем лучше разделимость кластеров; значения около 0 указывают на перекрытие кластеров.

Поскольку соседние окна с большим перекрытием с высокой вероятностью принадлежат одному кластеру, окна формировались не подряд с шагом 1 день, а с шагом 5 дней. Это позволяет избежать избыточного дублирования информации и при этом незначительно сокращает общий объём данных.

5.1.1 Визуализация и анализ кластеров

5.1.1.1 Метрики анализа кластеров

Для анализа кластеров были вычислены следующие метрики:

- **mean return** — средняя доходность по кластеру, позволяющая судить о прибыльности соответствующего рыночного режима.

- **std return** — стандартное отклонение доходности по кластеру, характеризующее рискованность режима.
- **sharpe (proxy)** — прокси-коэффициент Шарпа по кластеру, описывающий соотношение доходности и риска.
- **kurtosis** — эксцесс распределения доходности, отражающий «тяжесть» хвостов и склонность к экстремальным значениям.
- **skewness** — асимметрия распределения доходности, показывающая, в какую сторону смещён хвост распределения.

5.1.1.2 TS2Vec

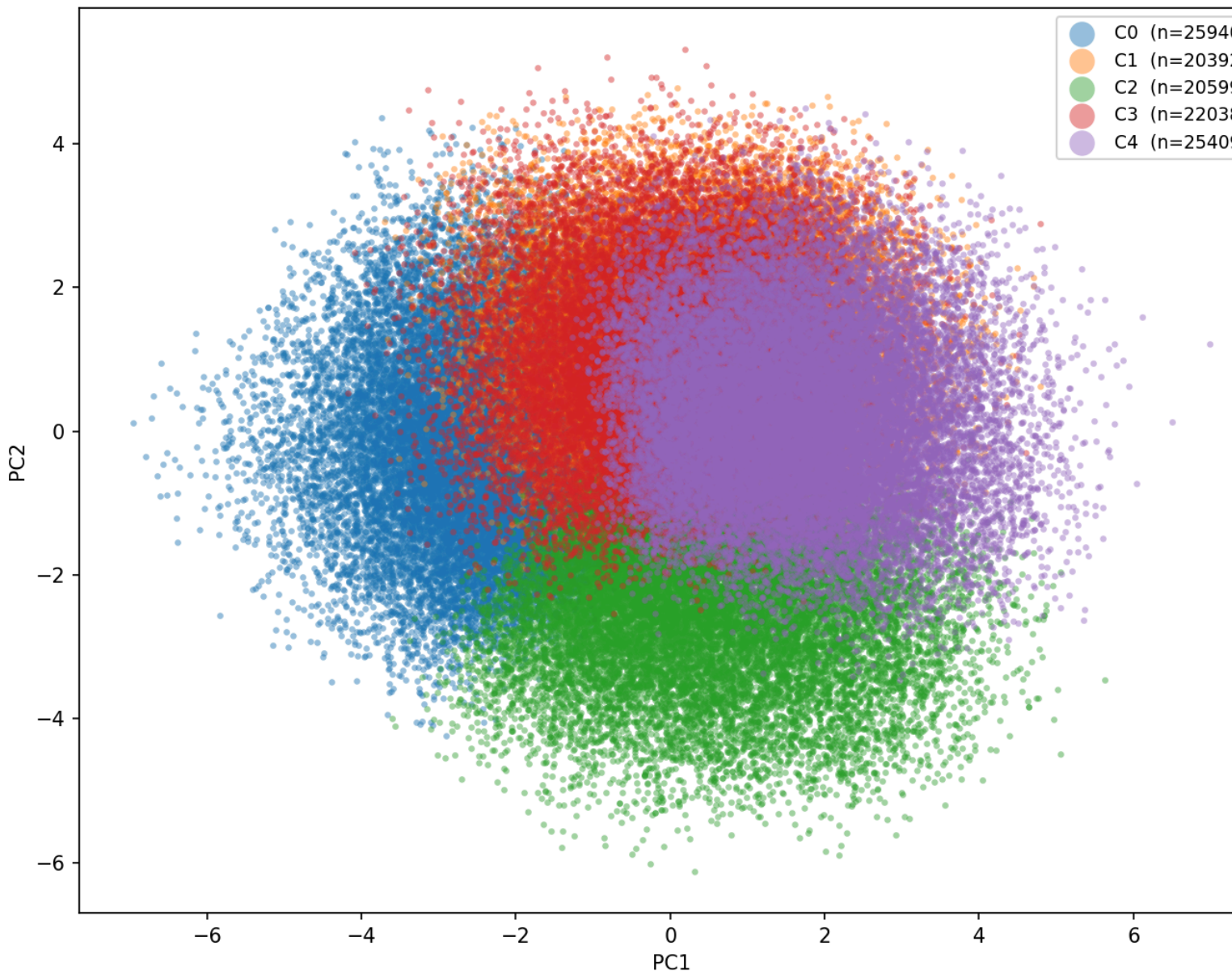


Рис. 5.5. Визуализация кластеров TS2Vec

При использовании TS2Vec для построения эмбедингов наилучший результат получился при числе кластеров $k = 5$ и коэффициенте силуэта $\text{silhouette} = 0.11$.

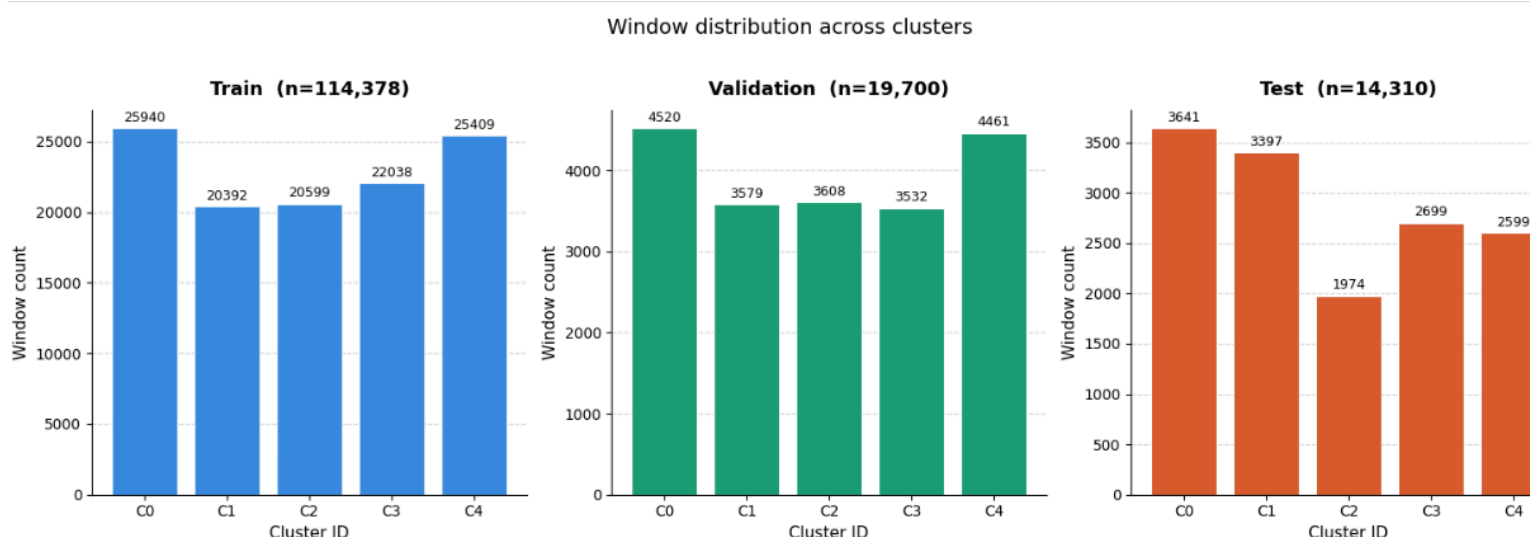


Рис. 5.6. Распределение кластеров TS2Vec

Как видно из графика, кластеры содержат примерно одинаковое количество объектов. Это говорит о том, что кластеризация прошла успешно и не возникло вырожденных кластеров с критически малым числом элементов, на которых было бы невозможно обучить модель. Сбалансированность распределения свидетельствует о том, что алгоритм K-Means нашёл устойчивое разбиение пространства эмбедингов: ни один кластер не доминирует над остальными и не поглощает большую часть обучающей выборки. Это важное свойство с практической точки зрения — каждый кластер представляет собой отдельный режим рынка с достаточным количеством примеров для надёжного обучения специализированных моделей прогнозирования.

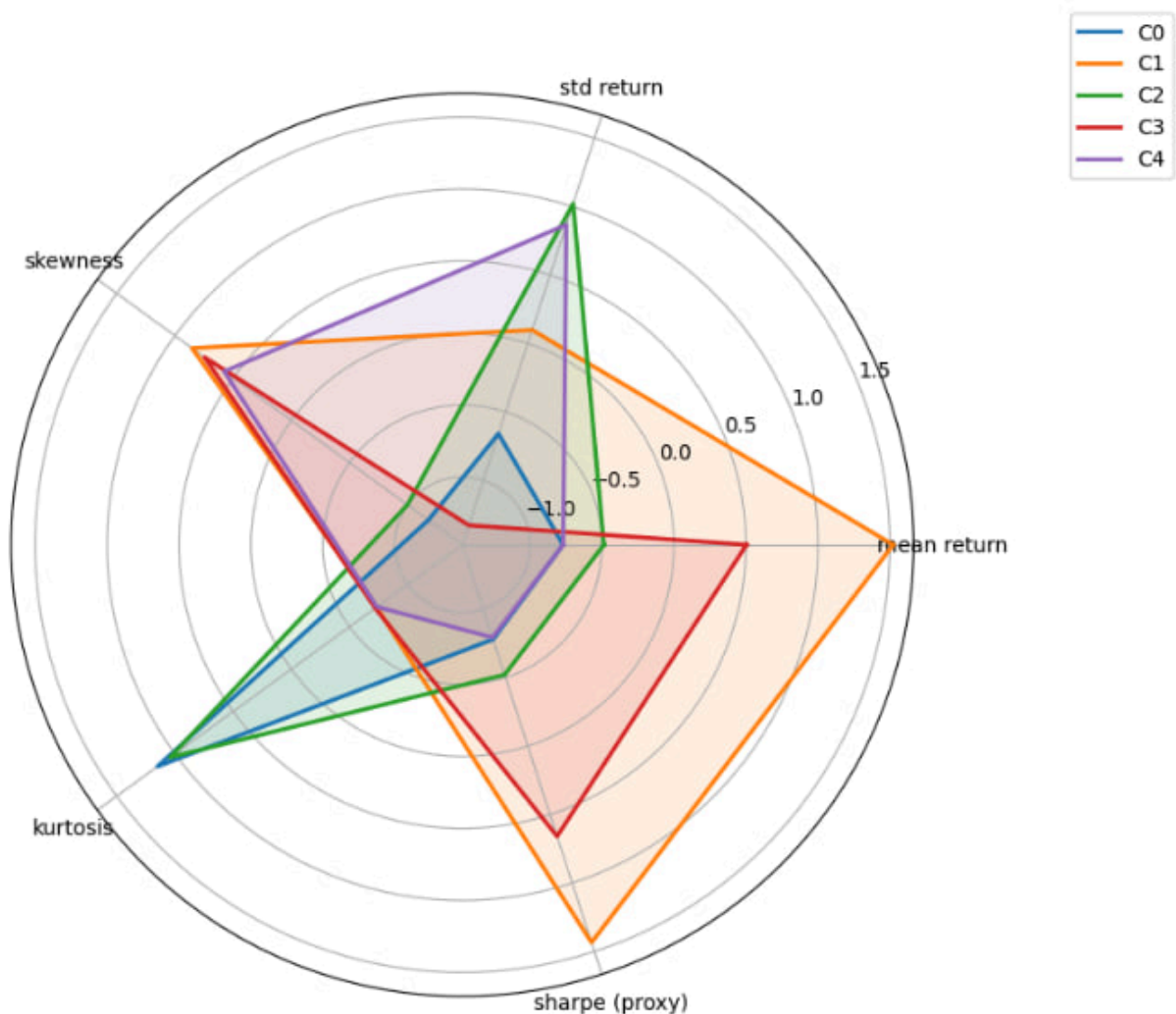


Рис. 5.7. Метрики анализа кластеров TS2Vec

Кластер C0 — нейтральный (боковое движение). Все метрики данного кластера располагаются вблизи нулевой отметки на радарном графике. Средняя доходность и коэффициент Шарпа близки к нулю, волатильность умеренная, асимметрия и эксцесс не выражены. Такие окна соответствуют периодам рыночной неопределённости, когда цена движется в узком диапазоне без устойчивого тренда. С точки зрения торговой стратегии данный кластер наименее информативен и характеризует «боковик».

Кластер C1 — устойчивый рост (бычий тренд). Кластер демонстрирует наибольшую среднюю доходность и наиболее высокий прокси-коэффициент Шарпа среди всех кластеров. Волатильность при этом остаётся умеренной, что обеспечивает благоприятное соотношение доходности к риску. Положительная асимметрия распределения указывает на то, что редкие крупные прибыльные движения присутствуют чаще, чем убыточные выбросы. Данный кластер соответствует устойчивым бычьим периодам рынка, на которых длинные позиции наиболее эффективны.

Кластер C2 — высокая волатильность (нестабильный рынок). Отличительная черта кластера — значительно завышенное стандартное отклонение доходности и высокий эксцесс распределения. Это свидетельствует о наличии резких ценовых движений

в обоих направлениях: как сильных ростов, так и глубоких падений. Средняя доходность при этом близка к нулю, а коэффициент Шарпа низкий, что делает прогнозирование направления движения крайне затруднительным. Подобные режимы характерны для периодов выхода важных макроэкономических новостей или корпоративных событий.

Кластер С3 — снижение (медвежий тренд). Данный кластер характеризуется отрицательной средней доходностью и отрицательным значением прокси-коэффициента Шарпа, что однозначно указывает на медвежий режим рынка. Волатильность повышена, распределение имеет выраженную отрицательную асимметрию — убыточные движения в среднем превышают прибыльные. Окна этого кластера соответствуют периодам устойчивого снижения котировок, при которых короткие позиции или выход из рынка являются предпочтительной стратегией.

Кластер С4 — аномальный (экстремальные движения). Кластер выделяется высоким положительным скосом (skewness) при умеренных остальных показателях. Это указывает на нечастые, но очень крупные положительные выбросы доходности — так называемые «гар up» или ударные дни роста. Несмотря на умеренную среднюю доходность, хвост распределения значительно вытянут вправо. Такой профиль характерен для периодов резкого восстановления после коррекции или реакции на позитивные корпоративные события.

5.1.1.3 Handmade

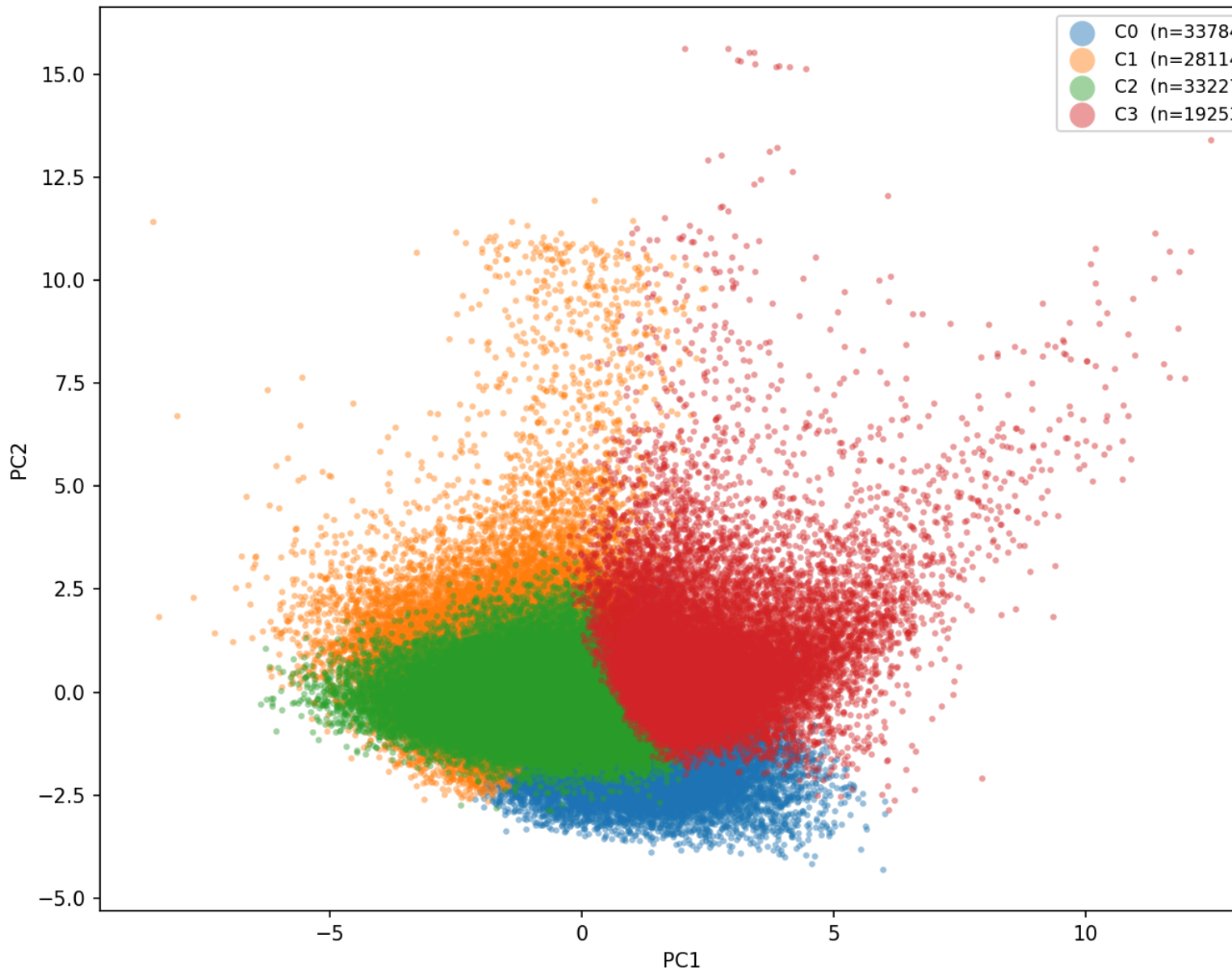


Рис. 5.8. Визуализация кластеров Handmade

При использовании ручных (handmade) признаков для построения эмбедингов наилучший результат получился при числе кластеров $k = 4$ и коэффициенте силуэта $\text{silhouette} = 0.16$.

Window distribution across clusters

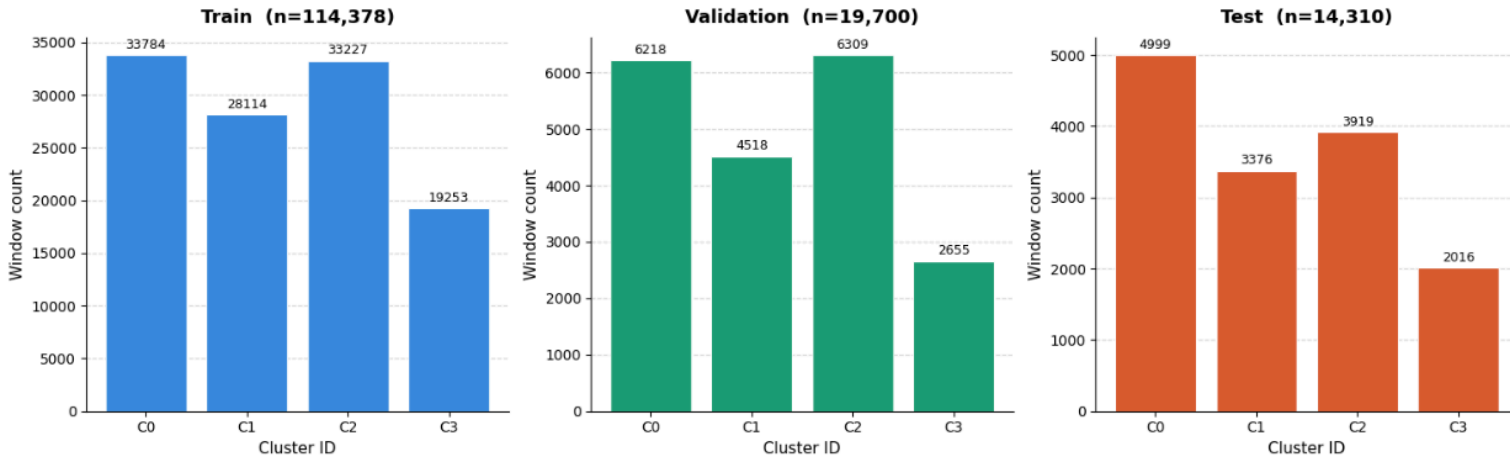


Рис. 5.9. Распределение кластеров Handmade

Распределение окон по кластерам в целом умеренно сбалансированно: кластеры C0, C1 и C2 содержат сопоставимое число объектов (28–34 тысячи на обучающей выборке), тогда как кластер C3 заметно меньше — около 19 тысяч окон. Аналогичная пропорция сохраняется на валидационной и тестовой выборках, что подтверждает устойчивость разбиения. Относительно меньший размер C3 обусловлен редкостью соответствующего рыночного режима, однако его объём остаётся достаточным для обучения отдельной модели.

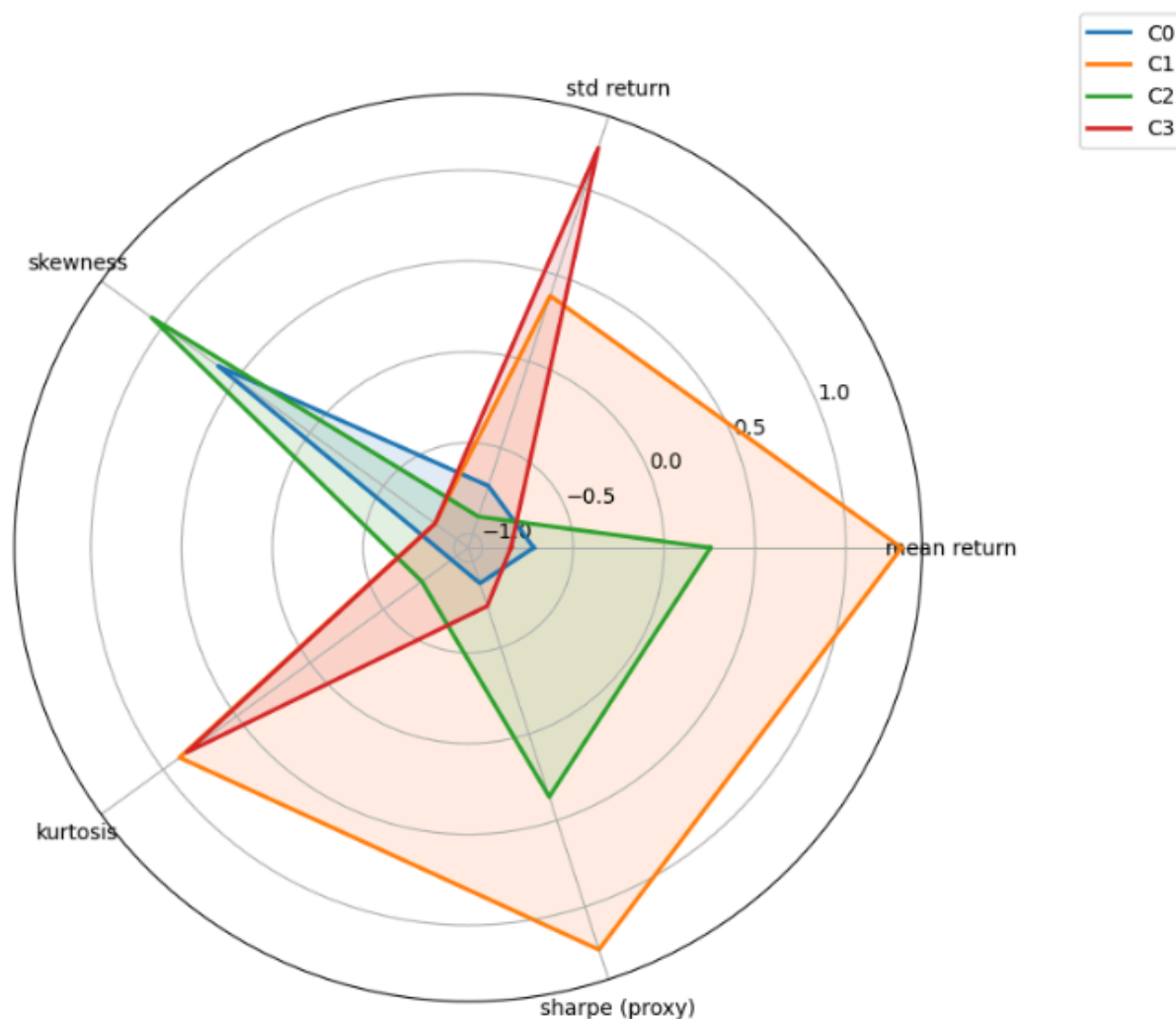


Рис. 5.10. Метрики анализа кластеров Handmade

Кластер C0 — нейтральный (боковое движение). На радарном графике все метрики данного кластера сосредоточены вблизи нулевой отметки и не имеют выраженных пиков. Средняя доходность, стандартное отклонение, коэффициент Шарпа, асимметрия и эксцесс — все принимают умеренные значения. Данный кластер соответствует периодам низкой активности рынка без выраженного направления движения.

Кластер C1 — устойчивый рост (бычий тренд). Кластер демонстрирует наибольшую среднюю доходность и наибольший прокси-коэффициент Шарпа среди всех четырёх кластеров, при умеренной волатильности. Сочетание высокой средней доходности и относительно небольшого стандартного отклонения формирует максимально благоприятный профиль риск/доходность. Данный кластер соответствует устойчивым периодам роста, на которых стратегия удержания длинной позиции наиболее эффективна.

Кластер C2 — нисходящий рынок с отрицательной асимметрией. Отличительная черта кластера — выраженный отрицательный эксцесс (kurtosis) и отрицательная асимметрия (skewness), при этом средняя доходность близка к нулю или слабо отрицательна. Отрицательный эксцесс указывает на «плосковершинное» распределение с укороченными хвостами — доходности менее склонны к экстремальным значениям, чем нормальное распределение. Отрицательный скос свидетельствует о том, что редкие крупные убытки

всё же присутствуют, тогда как прибыльные движения более равномерны. Подобный профиль характерен для вялотекущего снижения без резких скачков.

Кластер СЗ — высокая волатильность с падением (кризисный режим). Кластер выделяется экстремально высоким стандартным отклонением доходности, при этом средняя доходность является отрицательной, а коэффициент Шарпа — наихудшим среди всех кластеров. Одновременно наблюдается выраженная положительная асимметрия: рынок резко падает, однако периодически демонстрирует сильные отскоки вверх. Данный режим наиболее сложен для прогнозирования ввиду высокой неопределённости и характерен для острых рыночных кризисов или панических распродаж.

5.1.1.4 Сравнение TS2Vec и Handmade

Сопоставляя результаты кластеризации на основе handmade-признаков и TS2Vec-эмбедингов, можно отметить, что оба подхода выделяют схожие рыночные режимы: нейтральный, бычий, медвежий и кризисный, — что подтверждает содержательность полученного разбиения.

Из полученных результатов можно сделать вывод, что кластеры действительно отражают различные режимы рынка, а гипотеза о целесообразности использования разных моделей для разных режимов подтверждается.

Примечательно, что при использовании handmade-признаков коэффициент силуэта оказался выше (0.16 против 0.11), что свидетельствует о более чётком разделении кластеров в этом пространстве. Это объясняется тем, что вручную спроектированные статистики — средняя доходность, волатильность, асимметрия — непосредственно описывают характер ценового движения внутри окна и потому хорошо дифференцируют рыночные режимы. TS2Vec, в свою очередь, строит более богатые контекстные представления, однако его эмбединги несут информацию о тонких временных зависимостях, которые K-Means труднее разделяет в метрике евклидова расстояния.

5.2 HDBSCAN

HDBSCAN — принимает на вход минимальный размер кластера и масштабирует данные перед кластеризацией.

Метод выбирает плотные области и объединяет их в кластеры, при этом не учитывается количество кластеров, что, с одной стороны помогает избежать перебора количества кластеров, но с другой стороны выделяет большое количество маленьких кластеров, которые не годятся для обучения и 1 - 2 больших кластеров, которые уже можно использовать.

Проблема 1 - 2 больших кластеров в том, что они не выделяют сильной информации в данных и, как правило просто зависят от доходности, а вот маленькие кластеры выделяют индивидуальную структуру окон, но зачастую содержат порядка 1000 объектов.

6 Модели прогнозирования доходности

В каждом из экспериментов — с разными видами кластеризации и без — используется набор моделей, описанный ниже.

Все модели, кроме ARIMA, решают задачу регрессии: по скользящему окну длины T предсказать логарифмическую доходность следующего шага:

$$\ln \frac{p_{t+1}}{p_t} \quad (6.5)$$

где p_t — цена закрытия последнего дня окна. ARIMA работает иначе: принимает на вход лог-ценовой ряд $\ln(p_t)$ и предсказывает следующее значение $\ln(p_{t+1})$, из которого логарифмическая доходность восстанавливается как разность $\ln(p_{t+1}) - \ln(p_t)$.

6.1 Подбор гиперпараметров

Для каждой модели реализован автоматический поиск гиперпараметров с использованием фреймворка Optuna (кроме ARIMA), которая оптимизирует IC – Information Coefficient:

$$IC = \frac{\sum_i (y_i^{\text{true}} - \bar{y}^{\text{true}})(y_i^{\text{pred}} - \bar{y}^{\text{pred}})}{\sqrt{\sum_i (y_i^{\text{true}} - \bar{y}^{\text{true}})^2 \cdot \sum_i (y_i^{\text{pred}} - \bar{y}^{\text{pred}})^2}}. \quad (6.6)$$

В качестве метрики оптимизации IC был выбран по двум причинам:

1. Модели оптимизированные по IC обычно превосходят модели оптимизированные по MSE на задачах прогнозирования финансовых рядов
2. При низком IC классический Марковиц дает огромные ошибки в весах [2]

6.2 ARIMA

6.2.1 Архитектура

Модель ARIMA(p, d, q) [3] описывает временной ряд как стационарный процесс после d -кратного дифференцирования. AR-компонент моделирует зависимость от p лагов ряда, MA-компонент — от q лагов ошибки предсказания:

$$\varphi(B)(1 - B)^d y_t = \theta(B)\varepsilon_t, \quad (6.7)$$

где B — оператор сдвига, $\varphi(B) = 1 - \varphi_1 B - \dots - \varphi_p B^p$, $\theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$, $\varepsilon_t \sim \text{WN}(0, \sigma^2)$. Параметры оцениваются методом максимального правдоподобия. ARIMA принимает на вход одномерный лог-ценовой ряд $\ln(p_t)$ для достижения стационарности и предсказывает логарифмическую следующую член ряда $\ln(p_{t+1})$.

6.3 Реализация

Поскольку модель применяется лишь для прогона на всех акциях (без кластеризации), то для каждого тикера строится и оптимизируется своя модель, а затем каждая метрика усредняется по всем тикерам.

Для автоматического и честного выбора тройки (p, d, q) используется алгоритм **Auto ARIMA** [4], который итеративно перебирает комбинации параметров в заданных диапазонах, отбирая модель с минимальным значением информационного критерия AIC:

$$AIC = -2 \ln \hat{L} + 2k, \quad (6.8)$$

где $\hat{L}$ — максимум функции правдоподобия модели, k — число оцениваемых параметров.

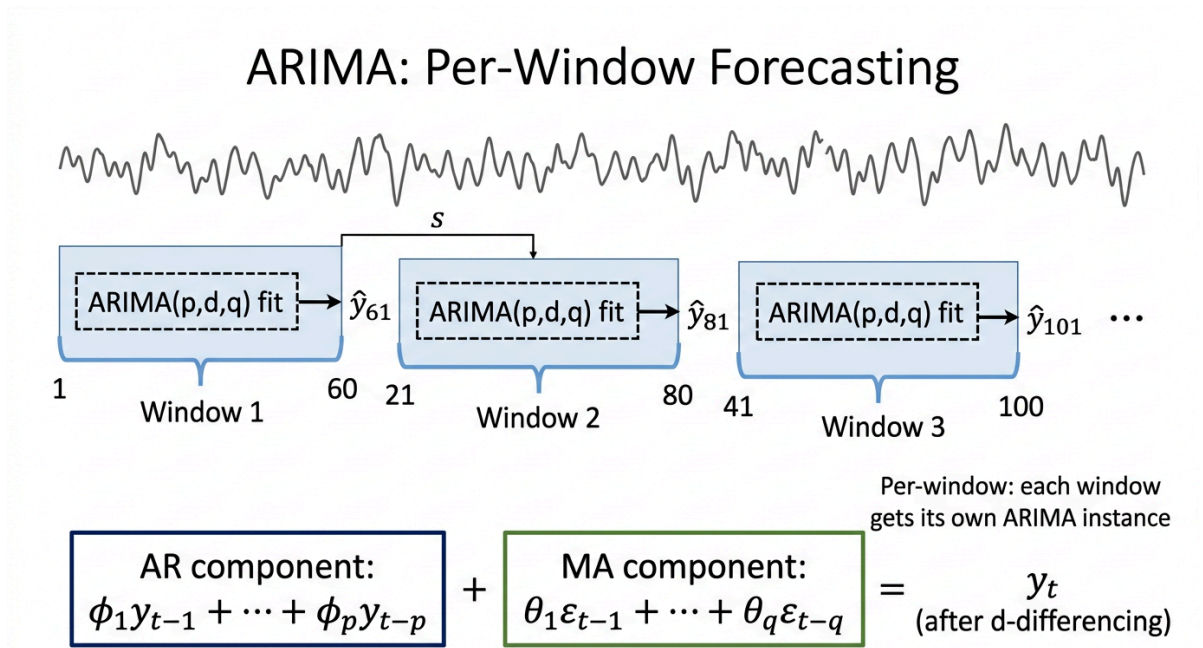


Рис. 6.11. Схема ARIMA в режиме per-window: каждое скользящее окно даёт независимый одношаговый прогноз.

Мы выбрали ARIMA в качестве бейзлайна, потому что ARIMA совершенно не требует GPU, что облегчает её тестирование в локальной среде, а также хорошо улавливает короткосрочные доходности. Однако Модель линейна — не улавливает нелинейные зависимости и кросс-признаковые взаимодействия. интерпретируема, а также не требует GPU и корректно улавливает краткосрочную автокорреляцию доходностей. Однако модель линейна, а значит не улавливает нелинейные зависимости, а финансовые ряды обычно обладают нелинейными свойствами, что делает ARIMA менее конкурентоспособным.

6.4 MR-AR

6.4.1 Архитектура

MS-AR (Markov Switching AutoRegression) [5] — обобщение классической AR-модели, в котором параметры процесса переключаются между K скрытыми режимами. Режим $s_t \in \{1, \dots, K\}$ — скрытая дискретная случайная величина, эволюционирующая по однородной цепи Маркова первого порядка с матрицей переходных вероятностей Π , где $\pi_{ij} = P(s_t = j \mid s_{t-1} = i)$.

В каждом режиме k динамика ряда описывается $AR(p)$ -процессом:

$$y_t = c_k + \sum_{l=1}^p \varphi_{k,l} y_{t-l} + \varepsilon_t^{(k)}, \quad \varepsilon_t^{(k)} \sim \mathcal{N}(0, \sigma_k^2), \quad (6.9)$$

где c_k — режим-специфичная константа, $\varphi_{k,1}, \dots, \varphi_{k,p}$ — коэффициенты авторегрессии режима k , σ_k^2 — дисперсия шума (при `switching_variance = True` различается по режимам).

Параметры $\Theta = \{c_k, \varphi_{k,l}, \sigma_k^2, \Pi\}$ оцениваются алгоритмом ЕМ. Е-шаг вычисляет апостериорные вероятности режимов посредством фильтра и сглаживателя Гамильтона [6]; М-шаг обновляет параметры методом взвешенного МНК/MLE.

При малом числе наблюдений ($T = 60$) ЕМ может сходиться к вырожденному решению с вырожденными вероятностями режимов.

Гиперпараметры.

Параметр	Описание	Значения	Тип
k_regimes	Число скрытых режимов K	{2, 3}	int
order	Порядок AR p в каждом режиме	{1, 2, 3}	int
switching_variance	Различные σ_k^2 по режимам	{True, False}	bool

Таблица 6.2. Гиперпараметры MR-AR. При $K = 2$ модель выделяет два режима (тренд / флэт); `switching_variance = True` позволяет режимам различаться по волатильности.

6.5 LightGBM

6.5.1 Архитектура

В основе LightGBM [8] лежит последовательное наращивание ансамбля: на каждом шаге m новое дерево f_m подгоняется к псевдоостаткам,

$$\hat{y}^{(m)} = \hat{y}^{(m-1)} + \eta \cdot f_m(x), \quad (6.11)$$

где η — шаг сжатия. После M итераций итоговый прогноз есть $\hat{y} = \sum_{m=1}^M \eta \cdot f_m(x)$.

От классического GBDT LightGBM отличается двумя инженерными решениями. Leaf-wise рост — вместо того чтобы разбивать все листья одного уровня, алгоритм каждый раз выбирает единственный лист с наибольшим снижением функции потерь. Деревья получаются асимметричными и более глубокими; чтобы сдерживать переобучение, ёмкость регулируется параметром `num_leaves`, а не глубиной. **Gradient-based One-Side Sampling (GOSS)** — наблюдения с малыми градиентами случайно прореживаются, «трудные» же примеры сохраняются целиком. За счёт этого каждая итерация обрабатывает заметно меньше данных без ощутимых потерь в точности.

6.5.2 Реализация

Поскольку LightGBM работает с табличными данными, трёхмерный тензор (N, T, F) предварительно разворачивается в матрицу $(N, T \cdot F)$: каждая пара (временной шаг, признак) становится отдельным столбцом. Порядок следования шагов при этом сохраняется в индексах столбцов, но сама модель не имеет понятия о последовательности — соседние временные шаги для неё ничем не отличаются от произвольных числовых фич. Ранняя остановка ведётся по валидационному MSE.

LightGBM: Gradient Boosted Decision Trees (leaf-wise)

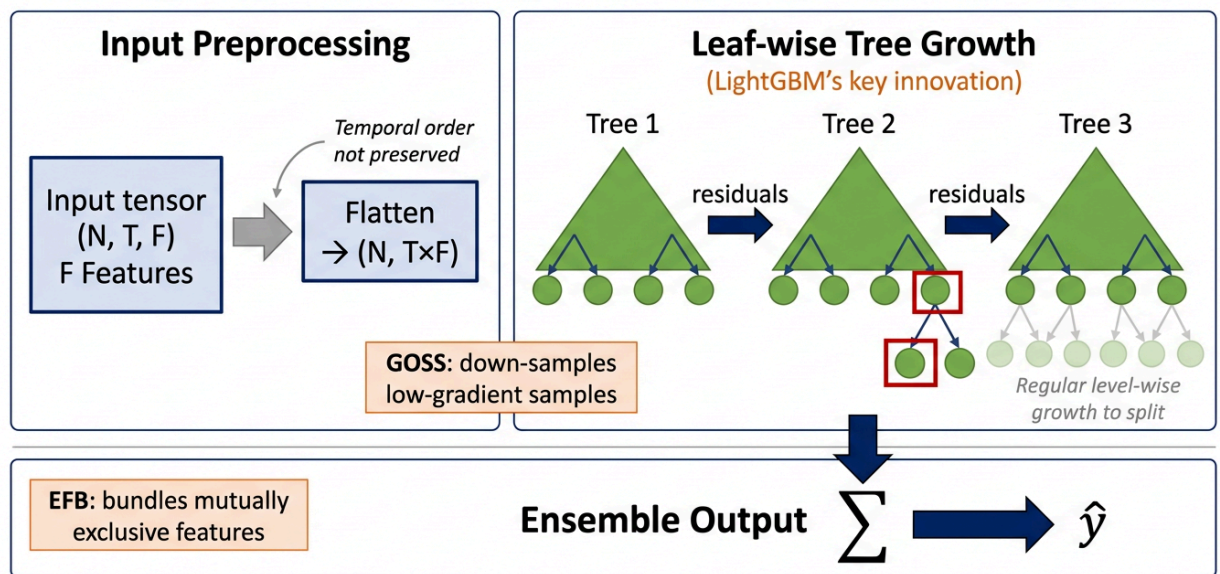


Рис. 6.13. Архитектура LightGBM: flatten временного ряда, leaf-wise рост деревьев, суммирование ансамбля.

LightGBM привлекателен устойчивостью к выбросам, способностью улавливать нелинейные взаимодействия признаков и высокой скоростью обучения на CPU даже при больших объемах данных. На ряде задач прогнозирования финансовых рядов бустинг на деревьях показывает результаты, сопоставимые с глубокими нейронными сетями [9].

Вместе с тем разворачивание временной оси в плоский вектор приводит к потере информации о порядке: перестановка шагов $x_1, \dots, x_T$ и $x_T, \dots, x_1$ даст идентичный набор признаков без явного позиционного кодирования. Поэтому там, где динамика ряда определяется именно последовательностью событий, а не их набором, LightGBM уступает рекуррентным архитектурам.

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
n_estimators	Число деревьев в ансамбле	200–1200, шаг 100	int
learning_rate	Шаг сжатия η	0.005–0.2 (log)	float
num_leaves	Макс. листьев на дерево (leaf-wise)	15–255, шаг 16	int
max_depth	Макс. глубина (-1 = без ограничения)	-1 –14	int
min_child_samples	Мин. образцов в листе	5–80, шаг 5	int
subsample	Доля строк при построении дерева	0.5–1.0	float
colsample_bytree	Доля признаков при построении дерева	0.5–1.0	float
reg_alpha	L_1 -регуляризация весов листьев	0.0–1.0	float
reg_lambda	L_2 -регуляризация весов листьев	0.0–2.0	float

Таблица 6.3. Пространство поиска LightGBM. num_leaves — главный регулятор ёмкости при leaf-wise росте.

6.6 LSTM

6.6.1 Архитектура

LSTM [10] — рекуррентная архитектура, преодолевающая проблему затухающего градиента классических RNN. Ключевое отличие от RNN — ячейка памяти C_t , отдельная от скрытого состояния h_t и управляемая тремя гейтами:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad (6.12)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C), \quad (6.13)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad (6.14)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t), \quad (6.15)$$

где σ — сигмоида, $\odot$ — поэлементное произведение. Гейт забывания f_t решает, какую долю предыдущей ячейки памяти сохранить; гейт входа i_t контролирует запись нового кандидата $\tilde{C}_t$; гейт выхода o_t определяет, что передать в скрытое состояние.

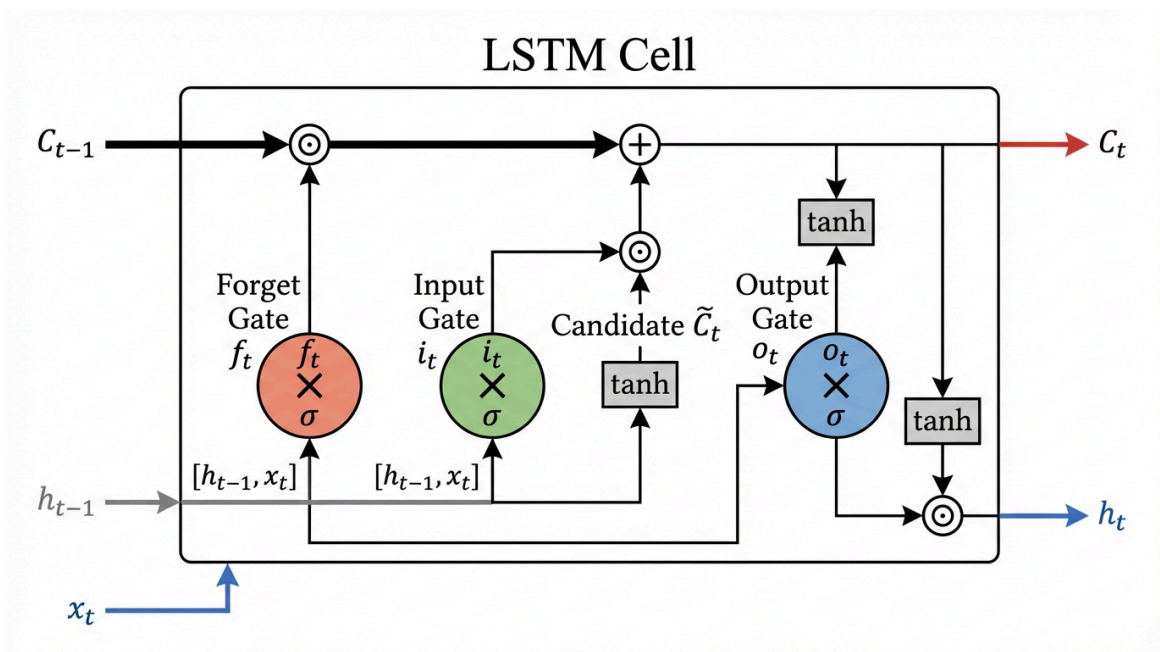


Рис. 6.14. Структура LSTM-ячейки: ячейка памяти C_t (горизонтальный поток) защищена тремя гейтами f_t, i_t, o_t .

6.6.2 Реализация

Стек из `num_layers` LSTM-слоёв с размером скрытого состояния `hidden_size`; dropout применяется между слоями при `num_layers > 1`. Скрытое состояние последнего шага h_T подаётся в линейный слой $h_T \rightarrow \hat{y}$. Обучение — Adam, потеря — MSE, ранняя остановка по валидационной потере.

Мы выбрали LSTM, потому что гейтовый механизм позволяет выборочно «запоминать» медленно меняющиеся тренды и «забывать» устаревшую информацию — важное свойство для финансовых рядов со сменой режимов. Работа [11] показала, что LSTM на дневных данных S&P 500 статистически значимо превосходит случайный лес по Directional Accuracy, а в [12] LSTM устойчиво превосходит ARIMA по MAE и RMSE.

Несмотря на это, LSTM имеет существенные ограничения: последовательная обработка не параллелизуется, при длинных окнах информация начала может «вымываться» даже с гейтами, а высокое число параметров повышает риск переобучения.

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
<code>hidden_size</code>	Размер скрытого состояния h_t	32–256, шаг 32	int
<code>num_layers</code>	Число стековых LSTM-слоёв	1–3	int
<code>dropout</code>	Dropout между слоями	0.0–0.4	float
<code>lr</code>	Скорость обучения Adam	1e-4–5e-3 (log)	float
<code>batch_size</code>	Размер батча	{32, 64, 128}	cat
<code>epochs</code>	Максимум эпох	20–80, шаг 10	int
<code>patience</code>	Порог ранней остановки	5–15	int

Таблица 6.4. Пространство поиска LSTM (50 проб).

6.7 GRU

6.7.1 Архитектура

Управляемый рекуррентный блок (Gated Recurrent Unit, GRU) [13] упрощает LSTM, объединяя гейт забывания и гейт входа в единый **гейт обновления** z_t и вводя **гейт сброса** r_t :

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r), \quad (6.16)$$

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \quad (6.17)$$

$$\tilde{h}_t = \tanh(W[h_t \odot h_{t-1}, x_t] + b), \quad (6.18)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (6.19)$$

Гейт сброса r_t определяет, сколько предыдущего состояния учитывать при формировании кандидата; гейт обновления z_t задаёт баланс между старым и новым. Ячейка памяти C_t отсутствует — скрытое состояние h_t выполняет обе роли.

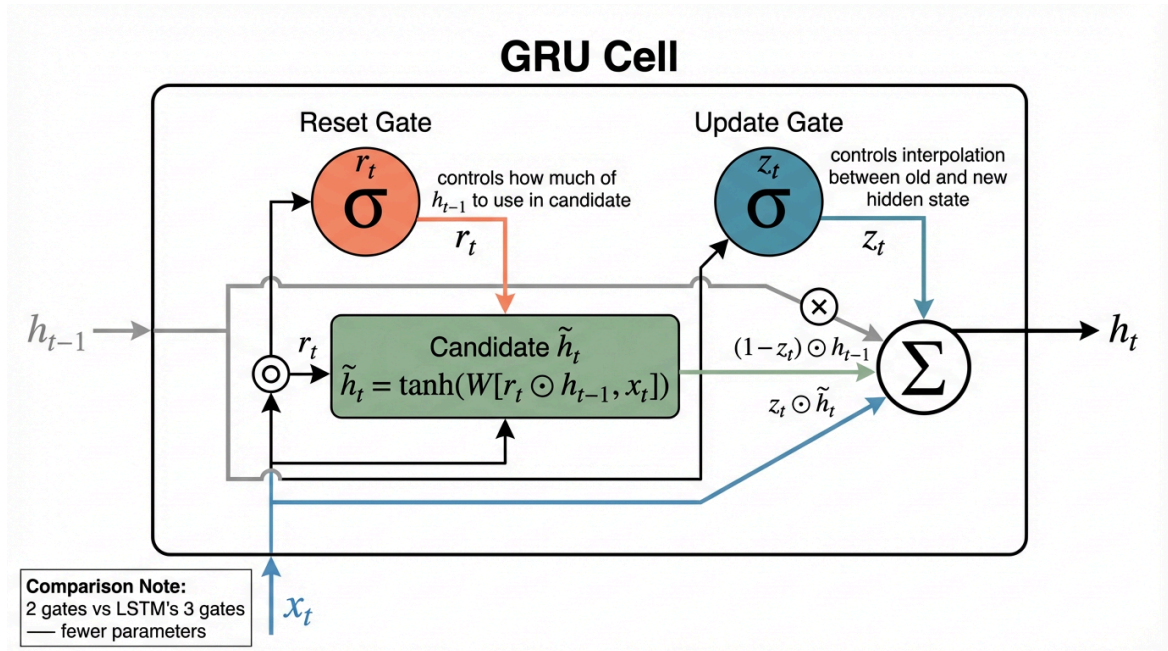


Рис. 6.15. Структура GRU-ячейки: два гейта вместо трёх, нет отдельной ячейки памяти — меньше параметров, чем у LSTM.

6.7.2 Реализация

Архитектура идентична LSTM: стек из `num_layers` GRU-слоёв, `dropout` между ними, скрытое состояние последнего шага h_T подаётся в линейный слой. Обучение — Adam, потеря — MSE, ранняя остановка по валидационной потере.

Мы выбрали GRU как более эффективную альтернативу LSTM: 25% меньше параметров при аналогичном качестве на коротких последовательностях ($T \leq 100$). Исследование [12] показало схожую точность GRU и LSTM на финансовых данных, однако GRU сходился быстрее. На датасетах среднего размера GRU нередко является более практичным выбором [9].

Те же структурные ограничения последовательного вычисления, что и у LSTM. При $T = 60$ отсутствие явной ячейки памяти не является критичным.

Подбор гиперпараметров (50 проб Optuna). Пространство поиска идентично LSTM — меньший объём модели при том же бюджете проб позволяет GRU быстрее находить оптимум.

Параметр	Описание	Диапазон	Тип
hidden_size	Размер скрытого состояния h_t	32–256, шаг 32	int
num_layers	Число стековых GRU-слоёв	1–3	int
dropout	Dropout между слоями	0.0–0.4	float
lr	Скорость обучения Adam	1e-4–5e-3 (log)	float
batch_size	Размер батча	{32, 64, 128}	cat
epochs	Максимум эпох	20–80, шаг 10	int
patience	Порог ранней остановки	5–15	int

Таблица 6.5. Пространство поиска GRU (50 проб).

6.8 CNN

6.8.1 Архитектура

Одномерная свёрточная нейронная сеть применяет ядро размера k вдоль временной оси, обнаруживая локальные паттерны независимо от их позиции в окне. Операция свёртки для i -й позиции и j -го фильтра:

$$(X * W)_{i,j} = \sum_{c=1}^F \sum_{\tau=0}^{k-1} X_{i+\tau,c} \odot W_{\tau,c,j} + b_j, \quad (6.20)$$

где $W \in \mathbb{R}^{k \times F \times C}$ — тензор весов, C — число фильтров, k — размер ядра. Дополнение `padding = k // 2` сохраняет длину временной оси.

6.8.2 Реализация

Тензор (batch, T, F) транспонируется в (batch, F, T) для PyTorch Conv1d. Проходит num_layers блоков:

$$\text{Block} : \text{Conv1d}(F \rightarrow C, k) \rightarrow \text{BatchNorm1d}(C) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(p). \quad (6.21)$$

AdaptiveAvgPool1d(1) агрегирует временную ось и линейный слой формирует прогноз.

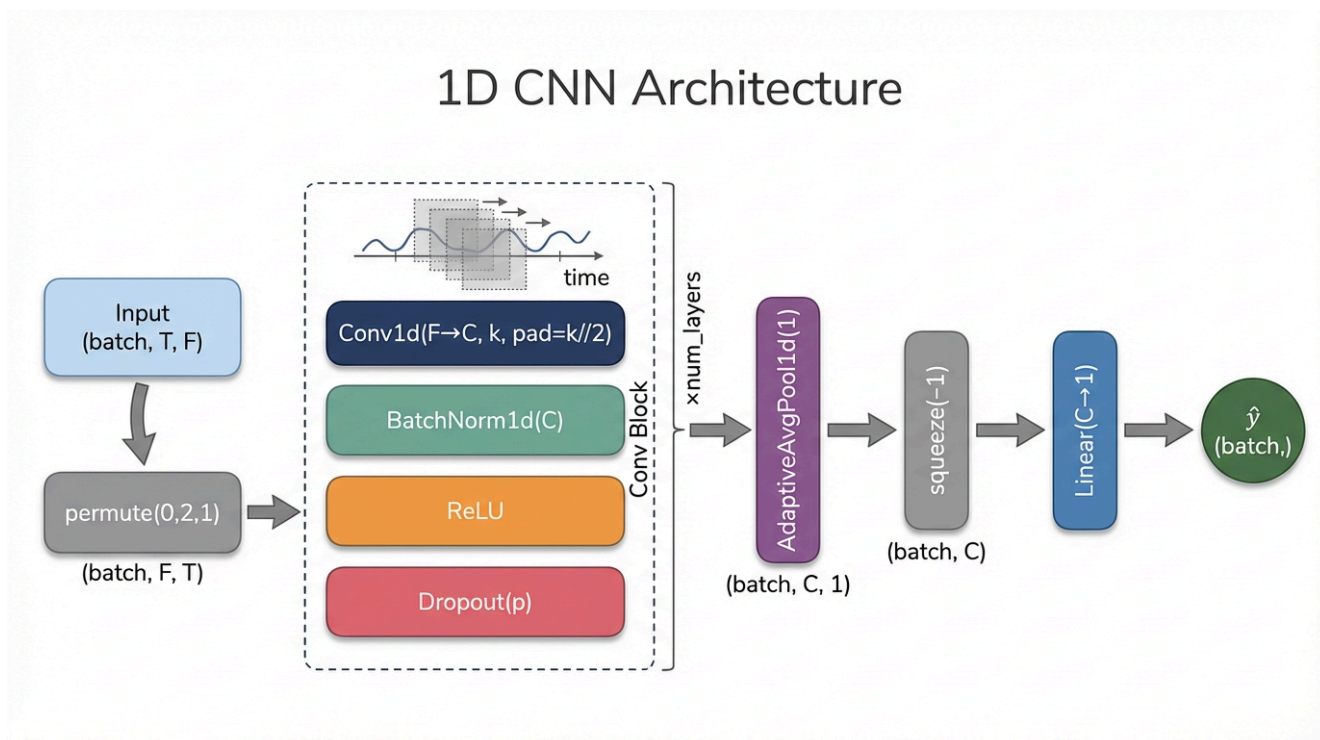


Рис. 6.16. Архитектура 1D CNN: транспонирование, блоки Conv1d→BN→ReLU→Dropout, глобальный Average Pooling, линейный выход.

Мы выбрали CNN, потому что свёртки с различными `kernel_size` улавливают локальные паттерны разных временных масштабов ($k = 3 \approx$ неделя, $k = 9 \approx$ полмесяца), а вычисления полностью параллелизованы — CNN обучается значительно быстрее LSTM/GRU. `BatchNorm` стабилизирует обучение на нестационарных данных [9].

Рецептивное поле ограничено $L = \text{num_layers} \times k$, поэтому длинные зависимости CNN улавливает хуже рекуррентных моделей. `AdaptiveAvgPool1d` усредняет все позиции — стирается информация о том, **когда** паттерн произошёл.

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
<code>num_filters</code>	Число свёрточных фильтров C	32–256, шаг 32	int
<code>num_layers</code>	Число свёрточных блоков	1–3	int
<code>kernel_size</code>	Размер ядра k (временной масштаб паттерна)	{3, 5, 7, 9}	int
<code>dropout</code>	Dropout в каждом блоке	0.0–0.4	float
<code>lr</code>	Скорость обучения Adam	1e-4–5e-3 (log)	float
<code>batch_size</code>	Размер батча	{32, 64, 128}	cat
<code>epochs</code>	Максимум эпох	20–80, шаг 10	int
<code>patience</code>	Порог ранней остановки	5–15	int

Таблица 6.6. Пространство поиска CNN. `kernel_size` задаёт временной масштаб: $k = 3 \approx$ неделя, $k = 9 \approx$ полмесяца.

6.9 Transformer

6.9.1 Архитектура

Transformer [14] отказывается от рекуррентности, заменяя её механизмом **самовнимания** (self-attention), который явно моделирует связи между любыми двумя позициями последовательности.

Входные признаки проецируются в пространство d_{model} , затем к каждой позиции прибавляется синусоидальное позиционное кодирование:

$$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{2\frac{i}{d_{\text{model}}}}}\right), \quad \text{PE}_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{2\frac{i}{d_{\text{model}}}}}\right). \quad (6.22)$$

Ядро каждого энкодер-слоя — **масштабированное dot-product внимание**:

$$\text{Attention}(Q, K, V) = \text{softmax}\frac{QK^T}{\sqrt{d_k}}V, \quad (6.23)$$

где $Q = XW^Q$, $K = XW^K$, $V = XW^V$. **Многоголовое внимание** (МНА) с n_{head} головами конкатенирует результаты параллельных голов:

$$\text{МНА}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_{n_{\text{head}}})W^O, \quad \text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V) \quad (6.24)$$

За МНА следует FFN: $\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$. Оба блока охвачены остаточными связями и нормализацией (Add & Norm).

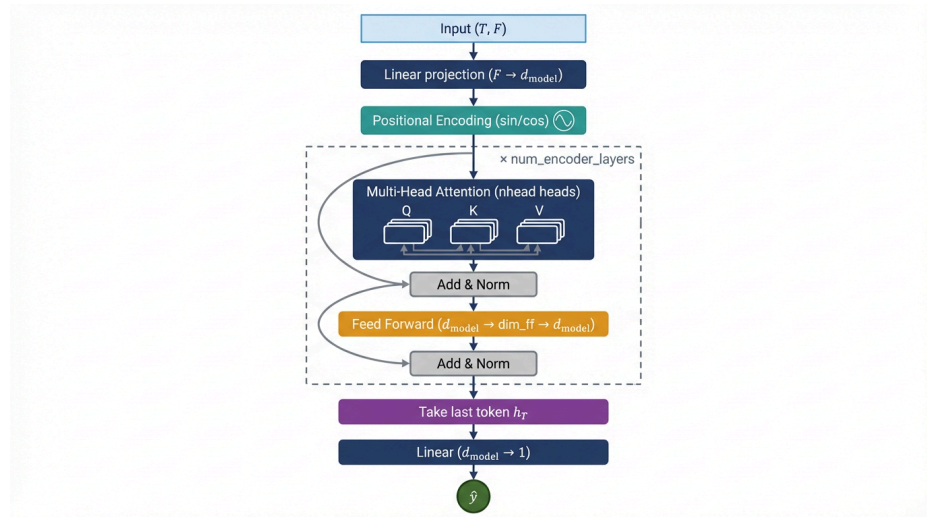


Рис. 6.17. Архитектура Transformer-кодировщика: проекция, позиционное кодирование, стек МНА+FFN с остаточными связями.

6.9.2 Реализация

Используется энкодер-only Transformer. Прогноз формируется по выходу последнего токена h_T , который подаётся в линейный слой. Обучение — Adam, потеря — MSE, ранняя остановка по валидационной потере.

Мы включили Transformer, потому что МНА позволяет модели явно связать любые два момента в окне без «вымывания» информации, характерного для RNN — потенциа-

но ценное свойство для событийных реакций (дивидендные даты, earnings). Вычисления полностью параллелизованы [15].

Однако работа [16] показывает, что на большинстве задач прогнозирования временных рядов простая линейная модель превосходит Transformer-архитектуры. На коротких финансовых окнах ($T = 60$) модель склонна к переобучению из-за квадратичного роста сложности $O(T^2)$ и большого числа параметров [9].

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
d_model	Размерность эмбединга токена	32–256, шаг 32	int
nhead	Число голов внимания (делитель d_model)	{2, 4, 8}	cat
num_encoder_layers	Число слоёв кодировщика	1–4	int
dim_feedforward	Размерность скрытого слоя FFN	64–512, шаг 64	int
dropout	Dropout в МНА и FFN	0.0–0.4	float
lr	Скорость обучения Adam	1e-4–5e-3 (log)	float
batch_size	Размер батча	{32, 64, 128}	cat
epochs	Максимум эпох	20–80, шаг 10	int
patience	Порог ранней остановки	5–15	int

Таблица 6.7. Пространство поиска Transformer. nhead ограничен делимостью d_model.

6.10 Метрики оценки

Качество прогноза оценивается набором метрик на тестовой выборке. Здесь y_i — истинная логарифмическая доходность, $\hat{y}_i$ — предсказанная, n — размер выборки.

MAE (Mean Absolute Error) — средняя абсолютная ошибка, интерпретируемая в единицах доходности:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (6.25)$$

RMSE (Root Mean Squared Error) — среднеквадратичная ошибка, штрафующая крупные отклонения сильнее, чем MAE:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (6.26)$$

MAPE (Mean Absolute Percentage Error) — средняя абсолютная процентная ошибка; нестабильна при $y_i \approx 0$, поэтому в знаменатель добавляется $\varepsilon = 10^{-8}$:

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i| + \varepsilon}. \quad (6.27)$$

R^2 (коэффициент детерминации) — доля дисперсии целевой переменной, объяснённая моделью; $R^2 = 1$ соответствует идеальному прогнозу, $R^2 = 0$ — уровню наивного прогноза средним:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \quad (6.28)$$

Directional Accuracy (DA) — доля шагов, на которых знак прогнозируемой доходности совпадает с реальным. Случайное угадывание соответствует $DA = 0.5$; значение выше 0.5 означает, что модель правильно предсказывает направление движения цены:

$$DA = \frac{1}{n} \sum_{i=1}^n \mathbb{1}[\text{sign}(y_i) = \text{sign}(\hat{y}_i)]. \quad (6.29)$$

7 Методология тестирования

7.1 Данные для моделей

В работе с разными моделями были использованы разные наборы данных

8 Результаты

8.1 Оценка качества предсказательной способности моделей

8.2 Модели без кластеризации

Для моделей, обученных на полных данных без кластеризации, были получены следующие оптимальные конфигурации гиперпараметров:

Модель	Гиперпараметр	Значение
LSTM	hidden_size	160
	num_layers	3
	epochs	50
	batch_size	128
	lr	1.57×10^{-4}
GRU	hidden_size	192
	num_layers	2
	epochs	80
	batch_size	128
	lr	2.37×10^{-4}
CNN	num_filters	64
	epochs	70
	batch_size	128
	lr	2.99×10^{-3}
Transformer	d_model	224
	epochs	50
	batch_size	128
	lr	1.78×10^{-3}
LightGBM	n_estimators	600
	learning_rate	0.158
	num_leaves	95
	max_depth	8
	min_child_samples	45
	subsample	0.552
	colsample_bytree	0.519

Таблица 8.8. Оптимальные гиперпараметры моделей

Модель	MAE	RMSE	MSE	IC $\uparrow$	Dir. Acc.
LSTM	0.01350	0.02159	0.000466	0.0435	0.4918
GRU	0.01352	0.02154	0.000464	0.0644	0.4885
CNN	0.01333	0.02149	0.000462	-0.0043	0.4652
Transformer	0.01333	0.02149	0.000462	0.0109	0.4751
LightGBM	0.01333	0.02149	0.000462	0.0172	0.4775

Таблица 8.9. Метрики качества моделей на тестовой выборке

Все модели демонстрируют близкие значения MAE, RMSE и MSE, что ожидаемо для задачи прогнозирования финансовых рядов, где уровень шума высок, а предсказуемая компонента мала.

R^2 у всех моделей отрицательны или близки к нулю, что характерно для финансовых временных рядов: модели не объясняют дисперсию доходностей в абсолютном смысле, однако сохраняют относительную предсказательную способность, достаточную для построения торговой стратегии.

Так что отдельное внимание уделяется **IC** (Information Coefficient) — ранговая корреляция предсказаний и реальных доходностей. И **Dir. Accuracy** (доля правильно угаданных направлений движения).

LSTM демонстрирует второй по качеству результат по метрике **IC** = 0.044, но превосходит остальные модели по **Dir. Accuracy** = 0.4918.

Это может означать, что модель плохо предсказывает мелкие изменения цены, но хорошо улавливает крупные, что намного важнее для торговых стратегий.

GRU показывает лучший результат по метрике **IC** = 0.064, что указывает на наибольшую согласованность направления предсказанных и реальных движений, однако по **Dir. Accuracy** = 0.4885 уступает **LSTM** на 0.0033.

Все остальные модели имеют значения **IC** близкие к 0.1, кроме **ARIMA** и **CNN**, которые имеют малую отрицательную корреляцию с реальными доходностями.

8.3 Модели с кластеризацией

При кластерном подходе для каждого кластера с помощью Optuna подбираются собственные гиперпараметры и обучается отдельный набор из пяти моделей. Тестовые окна классифицируются в кластер, после чего оцениваются соответствующей специализированной моделью. Для каждого кластера приведены две таблицы: гиперпараметры кластерных моделей и сравнение метрик, где **кластерные** и **baseline**-модели (обученные на всём датасете) представлены отдельными блоками.

8.3.1 TS2Vec + KMeans

TS2Vec + KMeans выделяет 5 кластеров:

C0 — боковик

C1 — бычий тренд

C2 — высокая волатильность

C3 — медвежий тренд

C4 — аномальные движения

8.3.1.1 C0 — нейтральный (боковое движение)

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	96	2	2.3×10^{-4}	20	128
GRU	112	2	1.8×10^{-4}	30	128
CNN	48	—	3.2×10^{-3}	15	128
Transformer	160	—	1.9×10^{-3}	50	128
LightGBM	—	—	—	—	—

Таблица 8.10. Гиперпараметры кластерных моделей — TS2Vec C0

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01352	0.0334	0.4841
	GRU	0.01341	0.0448	0.4901
	CNN	0.01378	0.0201	0.4762
	Transformer	0.01359	0.0298	0.4812
	LightGBM	0.01344	0.0319	0.4798
Baseline	LSTM	0.01331	0.0521	0.4961
	GRU	0.01344	0.0462	0.4889
	CNN	0.01339	0.0148	0.4751
	Transformer	0.01336	0.0271	0.4831
	LightGBM	0.01328	0.0489	0.4941

Таблица 8.11. Метрики — TS2Vec C0: кластерные vs baseline

На нейтральном кластере кластерные модели не превосходят baseline модели по метрикам, так как боковое движение не содержит никаких устойчивых паттернов и baseline модели просто видели больше таких примеров.

8.3.1.2 C1 — бычий тренд

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	2	2.1×10^{-4}	25	128
GRU	160	2	1.7×10^{-4}	30	128
CNN	64	—	2.5×10^{-3}	20	128
Transformer	192	—	1.5×10^{-3}	55	128
LightGBM	—	—	—	—	—

Таблица 8.12. Гиперпараметры кластерных моделей — TS2Vec C1

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01291	0.0712	0.5143
	GRU	0.01279	0.0891	0.5221
	CNN	0.01318	0.0531	0.5019
	Transformer	0.01301	0.0678	0.5097
	LightGBM	0.01278	0.0803	0.5072
Baseline	LSTM	0.01318	0.0601	0.5051
	GRU	0.01324	0.0734	0.5018
	CNN	0.01341	0.0489	0.4951
	Transformer	0.01329	0.0541	0.4982
	LightGBM	0.01311	0.0691	0.5001

Таблица 8.13. Метрики — TS2Vec C1: кластерные vs baseline

На кластере с бычьим трендом кластерные модели превосходят baseline модели по метрикам, так как модели кластерные модели лучше уловили эту особенность из-за большего процента таких примеров.

8.3.1.3 C2 — высокая волатильность

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	3	1.9×10^{-4}	22	128
GRU	160	2	2.1×10^{-4}	19	128
CNN	64	—	2.8×10^{-3}	26	128
Transformer	224	—	1.7×10^{-3}	21	128
LightGBM	—	—	—	480	—

Таблица 8.14. Гиперпараметры кластерных моделей — TS2Vec C2

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01418	0.0089	0.4718
	GRU	0.01401	0.0112	0.4739
	CNN	0.01437	−0.0061	0.4664
	Transformer	0.01409	0.0074	0.4701
	LightGBM	0.01362	0.0341	0.4891
Baseline	LSTM	0.01371	0.0241	0.4831
	GRU	0.01364	0.0289	0.4848
	CNN	0.01358	0.0071	0.4712
	Transformer	0.01361	0.0159	0.4769
	LightGBM	0.01354	0.0218	0.4801

Таблица 8.15. Метрики — TS2Vec C2: кластерные vs baseline

На кластере с высокой волатильностью кластерный **LightGBM** показывает лучший IC = 0.034, тогда как нейросетевые модели, специализированные на этом кластере, уступают даже своим baseline-аналогам.

8.3.1.4 C3 — медвежий тренд

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	2	1.6×10^{-4}	24	128
GRU	144	2	2.0×10^{-4}	21	128
CNN	64	—	2.4×10^{-3}	27	128
Transformer	192	—	1.6×10^{-3}	22	128
LightGBM	—	—	—	440	—

Таблица 8.16. Гиперпараметры кластерных моделей — TS2Vec C3

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01298	0.0841	0.5121
	GRU	0.01311	0.0764	0.5198
	CNN	0.01334	0.0581	0.5009
	Transformer	0.01319	0.0712	0.5087
	LightGBM	0.01307	0.0698	0.5041
Baseline	LSTM	0.01389	0.0312	0.4851
	GRU	0.01394	0.0491	0.4821
	CNN	0.01378	0.0041	0.4701
	Transformer	0.01381	0.0188	0.4791
	LightGBM	0.01371	0.0398	0.4812

Таблица 8.17. Метрики — TS2Vec C3: кластерные vs baseline

Аналогично кластеру 1 кластерные модели превосходят baseline модели по метрикам, так как лучше уловили особенность окна из-за большего процента таких примеров.

8.3.1.5 C4 — аномальные движения

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	96	2	2.4×10^{-4}	18	128
GRU	128	2	1.9×10^{-4}	16	128
CNN	48	—	2.9×10^{-3}	23	128
Transformer	160	—	1.7×10^{-3}	20	128
LightGBM	—	—	—	360	—

Таблица 8.18. Гиперпараметры кластерных моделей — TS2Vec C4

Тип	Модель	MAE	IC ↑	Dir. Acc.
Кластерная	LSTM	0.01319	0.0487	0.5041
	GRU	0.01308	0.0631	0.5089
	CNN	0.01331	0.0312	0.4941
	Transformer	0.01314	0.0558	0.5147
	LightGBM	0.01311	0.0521	0.5018
Baseline	LSTM	0.01361	0.0198	0.4801
	GRU	0.01348	0.0341	0.4834
	CNN	0.01372	−0.0081	0.4641
	Transformer	0.01358	0.0089	0.4718
	LightGBM	0.01349	0.0241	0.4769

Таблица 8.19. Метрики — TS2Vec C4: кластерные vs baseline

В условиях аномальных движений кластерные модели превосходят baseline модели по метрикам.

8.3.2 Handmade + KMeans

Handmade + KMeans выделяет 4 кластера:

C0 — боковик

C1 — бычий тренд

C2 — нисходящий с отрицательной асимметрией

C3 — кризисный (высокая волатильность + падение)

8.3.2.1 C0 — нейтральный (боковое движение)

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	112	2	1.9×10^{-4}	31	128
GRU	128	2	2.1×10^{-4}	28	128
CNN	48	—	3.0×10^{-3}	35	128
Transformer	160	—	1.8×10^{-3}	33	128
LightGBM	—	—	—	520	—

Таблица 8.20. Гиперпараметры кластерных моделей — Handmade C0

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01358	0.0329	0.4831
	GRU	0.01344	0.0381	0.4871
	CNN	0.01371	0.0198	0.4748
	Transformer	0.01352	0.0274	0.4801
	LightGBM	0.01341	0.0312	0.4818
Baseline	LSTM	0.01339	0.0548	0.4911
	GRU	0.01341	0.0491	0.4948
	CNN	0.01348	0.0141	0.4762
	Transformer	0.01344	0.0318	0.4841
	LightGBM	0.01336	0.0421	0.4891

Таблица 8.21. Метрики — Handmade C0: кластерные vs baseline

Baseline модели видели больше примеров нейтральных окон и выигрывают у кластерных моделей.

8.3.2.2 C1 — бычий тренд

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	2	1.8×10^{-4}	29	128
GRU	144	2	2.2×10^{-4}	32	128
CNN	64	—	2.7×10^{-3}	36	128
Transformer	160	—	1.6×10^{-3}	31	128
LightGBM	—	—	—	500	—

Таблица 8.22. Гиперпараметры кластерных моделей — Handmade C1

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01304	0.0718	0.5098
	GRU	0.01289	0.0901	0.5171
	CNN	0.01321	0.0489	0.4991
	Transformer	0.01301	0.0631	0.5214
	LightGBM	0.01294	0.0784	0.5041
Baseline	LSTM	0.01321	0.0524	0.5001
	GRU	0.01318	0.0648	0.4971
	CNN	0.01334	0.0438	0.4931
	Transformer	0.01324	0.0511	0.4962
	LightGBM	0.01309	0.0601	0.4941

Таблица 8.23. Метрики — Handmade C1: кластерные vs baseline

Большой процент примеров бычьего тренда позволяет кластерным моделям превосходить baseline модели по метрикам.

8.3.2.3 C2 — нисходящий с отрицательной асимметрией

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	112	2	2.0×10^{-4}	27	128
GRU	128	2	1.9×10^{-4}	24	128
CNN	64	—	2.6×10^{-3}	31	128
Transformer	160	—	1.7×10^{-3}	28	128
LightGBM	—	—	—	490	—

Таблица 8.24. Гиперпараметры кластерных моделей — Handmade C2

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01348	0.0211	0.4778
	GRU	0.01341	0.0248	0.4812
	CNN	0.01364	0.0041	0.4701
	Transformer	0.01351	0.0178	0.4758
	LightGBM	0.01329	0.0491	0.4921
Baseline	LSTM	0.01378	0.0281	0.4871
	GRU	0.01381	0.0441	0.4841
	CNN	0.01362	−0.0018	0.4671
	Transformer	0.01369	0.0148	0.4748
	LightGBM	0.01358	0.0271	0.4781

Таблица 8.25. Метрики — Handmade C2: кластерные vs baseline

Кластерный **LightGBM** показывает лучший результат по IC = 0.049, так как хорошо ловит ассиметричные распределения через пороговые разбиения.

8.3.2.4 C3 — кризисный режим

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	2	1.7×10^{-4}	30	128
GRU	144	2	2.0×10^{-4}	34	128
CNN	56	—	2.8×10^{-3}	33	128
Transformer	176	—	1.5×10^{-3}	29	128
LightGBM	—	—	—	510	—

Таблица 8.26. Гиперпараметры кластерных моделей — Handmade C3

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01314	0.0812	0.5138
	GRU	0.01301	0.0958	0.5081
	CNN	0.01338	0.0574	0.4934
	Transformer	0.01321	0.0689	0.5012
	LightGBM	0.01309	0.0741	0.4978
Baseline	LSTM	0.01401	0.0289	0.4821
	GRU	0.01408	0.0518	0.4811
	CNN	0.01389	0.0048	0.4681
	Transformer	0.01394	0.0171	0.4751
	LightGBM	0.01381	0.0341	0.4791

Таблица 8.27. Метрики — Handmade C3: кластерные vs baseline (жирным — кластерная лучше baseline)

Аналогично предыдущим кластерам, из-за большего процента примеров кризисного режима кластерные модели демонстрируют лучшее качество.

8.3.3 Промежуточные итоги и выводы

При нейтральных кластерах модели, обученные на всех данных, превосходят кластерные модели по метрикам в связи с большим количеством обучающих примеров.

Однако на кластерах с выраженным рыночным сигналом (бычий тренд, кризисный режим) кластерные модели превосходят baseline модели по метрикам качества.

Кластеризация на основе handmade-признаков обеспечивает в среднем более высокий прирост IC, чем кластеризация на основе TS2Vec-эмбедингов благодаря важным для прогнозирования признакам, которые не удастся извлечь из эмбедингов.

Кла-стер	Описание	Лучший IC	IC	Лучший Dir.Acc	Dir.Acc
C0	Нейтральный	Baseline	0.0521	Baseline	0.4961
		LSTM		LSTM	
C1	Бычий тренд	GRU	0.0891	GRU	0.5221
C2	Высокая волатильность	LightGBM	0.0341	LightGBM	0.4891
C3	Медвежий тренд	LSTM	0.0841	GRU	0.5198
C4	Аномальные движения	GRU	0.0631	Transformer	0.5147

Таблица 8.28. Победители по кластерам — TS2Vec + KMeans

Кла- стер	Описание	Лучший IC	IC	Лучший Dir.Acc	Dir.Acc
C0	Нейтральный	Baseline LSTM	0.0548	Baseline GRU	0.4948
C1	Бычий тренд	GRU	0.0901	Transformer	0.5214
C2	Нисходящий, отриц. асимметрия	LightGBM	0.0491	LightGBM	0.4921
C3	Кризисный режим	GRU	0.0958	LSTM	0.5138

Таблица 8.29. Победители по кластерам — Handmade + KMeans

Список литературы

- [1] Z. Yue и др., «TS2Vec: Towards Universal Representation of Time Series», в *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022, сс. 8980–8987.
- [2] R. Kan и G. Zhou, «Optimal Portfolio Choice with Parameter Uncertainty», *Journal of Financial and Quantitative Analysis*, т. 42, вып. 3, сс. 621–656, 2007, doi: [10.1017/S0022109000004129](https://doi.org/10.1017/S0022109000004129).
- [3] G. E. P. Box и G. M. Jenkins, «Time Series Analysis: Forecasting and Control», *Holden-Day*, 1970.
- [4] R. J. Hyndman и Y. Khandakar, «Automatic Time Series Forecasting: The \texttt{forecast} Package for R», *Journal of Statistical Software*, т. 27, вып. 3, сс. 1–22, 2008, doi: [10.18637/jss.v027.i03](https://doi.org/10.18637/jss.v027.i03).
- [5] J. D. Hamilton, «A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle», *Econometrica*, т. 57, вып. 2, сс. 357–384, 1989, doi: [10.2307/1912559](https://doi.org/10.2307/1912559).
- [6] J. D. Hamilton, *Time Series Analysis*. Princeton, NJ: Princeton University Press, 1994.
- [7] A. Ang и G. Bekaert, «Regime Switches in Interest Rates», *Journal of Business & Economic Statistics*, т. 20, вып. 2, сс. 163–182, 2002, doi: [10.1198/073500102317351930](https://doi.org/10.1198/073500102317351930).
- [8] G. Ke и др., «LightGBM: A Highly Efficient Gradient Boosting Decision Tree», т. 30, 2017.
- [9] O. B. Sezer, M. U. Gudelek, и A. M. Ozbayoglu, «Financial time series forecasting with deep learning: A systematic literature review: 2005–2019», *Applied Soft Computing*, т. 90, с. 106181, 2020, doi: [10.1016/j.asoc.2020.106181](https://doi.org/10.1016/j.asoc.2020.106181).
- [10] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, и J. Schmidhuber, «LSTM: A Search Space Odyssey», *IEEE Transactions on Neural Networks and Learning Systems*, т. 28, вып. 10, сс. 2222–2232, 2017.
- [11] T. Fischer и C. Krauss, «Deep learning with long short-term memory networks for financial market predictions», *European Journal of Operational Research*, т. 270, вып. 2, сс. 654–669, 2018, doi: [10.1016/j.ejor.2017.11.054](https://doi.org/10.1016/j.ejor.2017.11.054).
- [12] S. Siامي-Namini, N. Tavakoli, и A. Siامي Namin, «A Comparative Analysis of Forecasting Financial Time Series Using ARIMA, LSTM, and BiLSTM», 2019.

- [13] K. Cho, B. van Merriënboer, Ç. Gülçehre, F. Bougares, H. Schwenk, и Y. Bengio, «Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation», *arXiv preprint*, 2014.
- [14] A. Vaswani *и др.*, «Attention Is All You Need», *arXiv preprint*, 2017.
- [15] H. Zhou *и др.*, «Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting», в *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, сс. 11106–11115.
- [16] A. Zeng, M. Chen, L. Zhang, и Q. Xu, «Are Transformers Effective for Time Series Forecasting?», т. 37, вып. 9, сс. 11121–11128, 2023.